



Tribhuvan University
Faculty of Humanities and Social Sciences

A PROJECT REPORT

ON

“PETPUJA”

Web-Based Food Ordering System with Recommendation

Submitted to:

Department of Humanities and Social Science

Nagarjuna College of Information Technology

Sankhamul, Lalitpur

In partial fulfillment of the requirement for the Bachelors in Computer Application

Submitted by:

Bijay Devkota (42902008)

Ronish Prajapati (42902092)

BCA 8th semester, 2077 Batch

Under the supervision of

Mr. Ramesh Singh Saud



Tribhuvan University

Faulty of Humanities and Social Sciences

Nagarjuna College of Information Technology

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Bijay Devkota** and **Ronish Prajapati** entitled “**PetPuja**” in partial fulfilment of the requirements for the degree of Bachelor in Computer Application is recommended for final evaluation.

Mr. Ramesh Singh Saud

Project Supervisor

Nagarjuna College of Information Technology

BCA Department



Tribhuvan University

Faulty of Humanities and Social Sciences

Nagarjuna College of Information Technology

LETTER OF APPROVAL

This is to certify that this project prepared by **Bijay Devkota** and **Ronish Prajapati** entitled “**PetPuja**” in partial fulfilment of the requirement for the bachelor’s degree in Computer Application has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

.....

Mr. Ramesh Singh Saud

Supervisor

BCA Department

Nagarjuna College of IT

Program Coordinator

BCA Department

Nagarjuna College of IT

.....

Internal Examiner

Nagarjuna College of IT

External Examiner

Tribhuvan University

ABSTRACT

This project builds a complete Web-Based Food Ordering System with Recommendation that focuses on making online food ordering simple, Secure and user friendly. The system allows users to create accounts, verify their email, log in, manage their profile, browse items, add products to their cart, and place orders using either online payment or cash on delivery. Admin users can manage products, control user access, and oversee all orders through a dedicated dashboard, ensuring smooth system operations.

A key feature of the application is its product recommendation module. This module uses the Apriori algorithm to analyse past delivered orders and discover which items are frequently bought together. Based on this, the system suggests relevant food items to users while they order. If there is not enough order data to generate recommendations, the system instead shows popular items, ensuring the recommendation feature always provides value. In addition, real-time notifications are sent to users when their order status changes, keeping them updated throughout the process.

Overall, this project aims to deliver a easy online food ordering experience by combining secure authentication, intelligent product recommendations, and efficient order management. The result is a user-friendly platform that benefits both customers and administrators.

Keywords: Apriori Algorithm, User Access Control, Order Management, Secure Authentication.

ACKNOWLEDGEMENT

We express our sincere gratitude to Nagarjuna College of IT, Sankhamul, Lalitpur, for providing invaluable resources and unwavering support throughout our project. The college's dedication to academic excellence has greatly enriched our learning experience, and we deeply appreciate the tools and opportunities provided.

A special acknowledgement goes to our respected supervisor, Mr. Ramesh Singh Saud, for his exceptional guidance and insightful feedback during the course of this project. His expertise and constant encouragement have been instrumental in shaping the success of our work. We also extend our heartfelt thanks to the faculty and staff members of Nagarjuna College of IT for their contributions to this project.

Finally, we thank our classmates and peers for fostering a supportive environment that motivated us to excel in our endeavours.

Sincerely,

Bijay Devkota

Ronish Prajapati

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION	ii
LETTER OF APPROVAL	iii
ABSTRACT	iv
ACKNOWLEDGEMENT	v
TABLE OF CONTENTS	vi
LIST OF ABBREVIATION	viii
LIST OF FIGURES	ix
LIST OF TABLES	x
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope and limitation	3
1.4.1 Scope	3
1.4.2 Limitations	4
1.5 Development Methodology.....	4
1.6 Report Organization	6
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	7
2.1 Background Study	7
2.2 Literature Review	7
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	9
3.1 System Analysis	9
3.1.1 Requirement Analysis	9
3.1.2 Feasibility Study	11
3.2 Analysis	13
3.2.1 Object modelling using Class Diagram:	14
3.2.2 Dynamic modelling using State Diagram	15
3.2.3 Dynamic modeling using Sequence Diagram	17
3.2.4 Process modelling using Activity Diagrams	19
CHAPTER 4: IMPLEMENTATION AND TESTING	21
4.1 Implementation	21
4.1.1 Analysis and Design Tools	21

4.1.2 Tools and Technologies	22
4.2 Implementation Details of Modules	23
4.3 Testing	55
4.3.1 Unit Testing	55
4.3.2 System Testing	61
4.4 Result Analysis.....	69
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS	75
5.1 Conclusion.....	75
5.2 Future Recommendations	75
REFERENCES	77
APPENDIX- SCREENSHOT	79

LIST OF ABBREVIATION

API	Application Programming Interface
CSS	Cascading Style Sheet
DB	Database
HTML	Hypertext Markup Language
JS	Javascript
JSON	JavaScript Object Notation
JWT	Json Web Token
OTP	One Time Password
PIN	Personal Identification Number
SMTP	Simple Mail Transfer Peotocol

LIST OF FIGURES

Figure 1: Use Case Diagram of Food Ordering System	10
Figure 2: Gantt chart of Project schedule	13
Figure 3 Class Diagram of Food Ordering System	14
Figure 4 State Diagram of Food Ordering System	16
Figure 5 Sequence Diagram of Food Ordering System	17
Figure 6: Activity Diagram of Food Ordering System	19
Figure 7: System Architecture of Online Food Ordering System....	Error! Bookmark not defined.
Figure 8: Refined Class Diagram of Food Ordering System	Error! Bookmark not defined.
Figure 9 Refinement of Sequence Diagram of Food Ordering System...	Error! Bookmark not defined.
Figure 10: Component Diagram	Error! Bookmark not defined.
Figure 11: Flowchart for Recommendation	Error! Bookmark not defined.
Figure 12: Admin Registration	79
Figure 13 : Otp to verify Email.....	79
Figure 14 : Admin Approval Request.....	80
Figure 15: Admin Login.....	80
Figure 16: Add New Menu Item	81
Figure 17: Manage New Menu Item	81
Figure 18: Add special items	82
Figure 19: Order Management.....	82
Figure 20: Sales Statistics	83
Figure 21: Query Details	83
Figure 22: Home Page.....	84
Figure 23: Menu	84
Figure 24: Checkout.....	85

LIST OF TABLES

Table 1: Estimated Project Schedule	13
Table 2: Popular Items Ranking Table	Error! Bookmark not defined.
Table 3 :Test case for unsuccessful registration for user.....	55
Table 4 :Test case for successful registration for user	56
Table 5 :Test case for unsuccessful login for user	57
Table 6 :Test Case for successful login for user	57
Table 7 :Test Case for successful registration for admin	58
Table 8 :Test Case for unsuccessful registration for admin.....	59
Table 9 :Test Case for unsuccessful login for admin	60
Table 10 :Test Case for successful login for admin	60
Table 11 :Test case for System Testing of admin panel	61
Table 12: Test case for System Testing of User panel	65

CHAPTER 1: INTRODUCTION

1.1 Introduction

The Web-Based Food Ordering System with Recommendation is a modern, platform designed to simplify and digitalize the process of ordering food from local restaurants. The system enables users to browse menus, add items to their cart, and place orders seamlessly through an intuitive and responsive interface. Developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), the platform integrates multiple modules such as user authentication, order management, payment processing, and a recommendation engine.

The key objective of this project is to provide users with a convenient and secure online ordering experience while helping owners efficiently manage orders and menu updates through an admin dashboard. The system supports both cash-on-delivery (COD) and online payments via the Stripe payment gateway, ensuring flexibility for customers. Real-time order tracking, status updates, analytics and email notifications are handled through the integration of Nodemailer (Gmail SMTP), keeping users informed throughout their order journey.

One of the distinguishing features of PetPuja is its intelligent recommendation module, which utilizes the Apriori algorithm to analyze past orders and suggest frequently purchased item combinations. This enhances the user experience by offering recommendations based on real-time data. Additionally, the system ensures secure communication through JWT-based authentication, protecting user credentials and order data.

Overall, PetPuja combines modern web technologies, intelligent recommendation logic, and a modular architecture to deliver a scalable and efficient online food ordering solution. The project demonstrates the effective use of full-stack development practices, database management, and algorithmic implementation to solve real-world problems in the food service industry.

1.2 Problem Statement

The current food ordering process in the restaurant is primarily manual and inefficient, leading to frequent errors, slow service, and limited customer convenience. [1] Customers must either visit the restaurant in person or place orders over the phone, which often results in communication gaps, missed orders, and longer wait times. [2] The absence of a digital system restricts the restaurant from offering real-time order tracking, secure online payments, reports, and data-driven recommendation. This situation highlights the need for a centralized and automated food ordering system tailored to the specific operational requirements of the restaurant.

[3] Although large-scale online platforms such as Pathao and foodmandu have introduced digital ordering solutions, these third-party systems are not suitable for single-restaurant use due to high commission rates, dependency on external servers, and lack of control over customer data.[4] Existing studies and small-scale implementations show that local restaurants often lack dedicated platforms that integrate essential functions like order management, payment processing, and recommendations within one environment. As a result, staff spend additional time managing orders manually, and customers experience reduced satisfaction due to limited accessibility and transparency in the ordering process.

If this problem remains unresolved, it will continue to affect customer experience, staff efficiency, and the restaurant's overall business performance. The ideal outcome is a dedicated, web-based food ordering system that allows customers to view the menu, customize their orders, make secure online payments, and receive instant updates on their order status. So, the Web-Based Food Ordering System with Recommendation “Trio-order”, built using the MERN stack (MongoDB, Express.js, React.js, and Node.js), integrates Stripe for payments, JWT authentication, and an Apriori-based recommendation engine to provide intelligent food suggestions. This solution bridges the gap between traditional manual operations and modern digital convenience, enabling the restaurant to improve service quality, operational efficiency, and customer satisfaction.

1.3 Objectives

The primary goal of this project is to develop a smart, web-based food ordering platform that enhances the efficiency, recommendation, and operational transparency of restaurant services. The system aims to address the limitations of traditional ordering methods by using technology to create a better customer experience. The major objectives of our system are as follows:

- i. To develop an admin dashboard for restaurant management to easily add, edit, or remove menu items and monitor daily sales reports.
- ii. To implement secure user authentication and authorization using JSON Web Tokens (JWT) to ensure that user accounts and transactions are protected from unauthorized access.
- iii. To implement a recommendation engine using the Apriori algorithm to suggest related food items based on previous orders, enhancing the customer's ordering experience.

1.4 Scope and limitation

1.4.1 Scope

This project focuses on the design and development of a Web-Based Food Ordering System that simplifies food ordering through web application. It aims to improve customer convenience and restaurant efficiency through tracking, secure payments, and intelligent recommendations. The system connects customers and restaurants through a single platform, reducing order errors and enhancing overall service quality.

System Development: The system will be built using the MERN stack (MongoDB, Express.js, React.js, Node.js). It will include user-friendly interfaces for customers, restaurants, and admins, with secure payments via Stripe and order tracking using WebSocket. This ensures fast, scalable, and efficient performance across all modules.

Recommendation Engine: The Apriori algorithm will be used to analyze order data and suggest related food items. This smart recommendation feature enhances user experience and supports restaurants in boosting sales through effective upselling.

System Architecture and Database: The system's three-tier architecture separates the presentation, application, and data layers to improve maintainability and scalability. It will

store user, menu, and order data securely while ensuring quick data access and system reliability.

Dashboards and Analytics: An Admin Dashboard will manage users and restaurants, while a Restaurant Dashboard will track orders, menus, and reports. Built-in analytics will provide insights into sales, order status, payment status and most popular food items.

1.4.2 Limitations

While the Web-Based Food Ordering System provides an efficient and modernized approach to food ordering, certain limitations exist due to scope boundaries and technical constraints. These limitations include:

Limited Delivery Management: The system does not include real-time delivery tracking using GPS or driver management features.

Dependence on Internet Connectivity: As a fully web-based application, the system's functionality relies on stable internet access. Poor network conditions may affect order placement, payment processing, and real-time updates.

No Multi-User Role Hierarchy: The system has two main roles Admin and Customer. Additional user roles (e.g., delivery staff, kitchen staff, or manager-level accounts) are not implemented in this version.

No Native Mobile Application: The current version is designed as a web-based platform optimized for browsers. Mobile app functionality and push notifications are not included in this phase but may be developed in future updates.

Limited Multi-Branch Functionality: The system is designed primarily for single-branch restaurant operations. Supporting multiple branches with synchronized data and order tracking would require additional database and API modifications.

1.5 Development Methodology

The development of the Web-Based Food Ordering System followed the Waterfall Software Development Model, which provides a structured, step-by-step approach suitable for projects with clearly defined goals. This model ensured that each phase was completed before moving to the next, allowing for organized planning, documentation, and implementation.

The process began with Requirement Analysis, where functional and non-functional needs were identified, including menu management, order tracking, payment, integration admin dashboard, and the recommendation system. The system is built in three tier architecture using the MERN stack (MongoDB, Express.js, React.js, Node.js), with, user interfaces, and workflows clearly defined.

During Implementation, the backend was developed using Node.js and Express.js with RESTful APIs for user authentication, orders, and payments. The frontend, created with React.js and Tailwind CSS, offered a smooth user experience, while Stripe was integrated for secure payments and the Apriori algorithm for intelligent food recommendations.

The Testing phase involved unit, integration, and user acceptance testing to ensure performance and reliability. After successful validation, the system was Deployed on a cloud platform for accessibility and scalability. Finally, in the Maintenance stage, minor bugs and updates were addressed, with future plans to expand features such as delivery and mobile app integration.

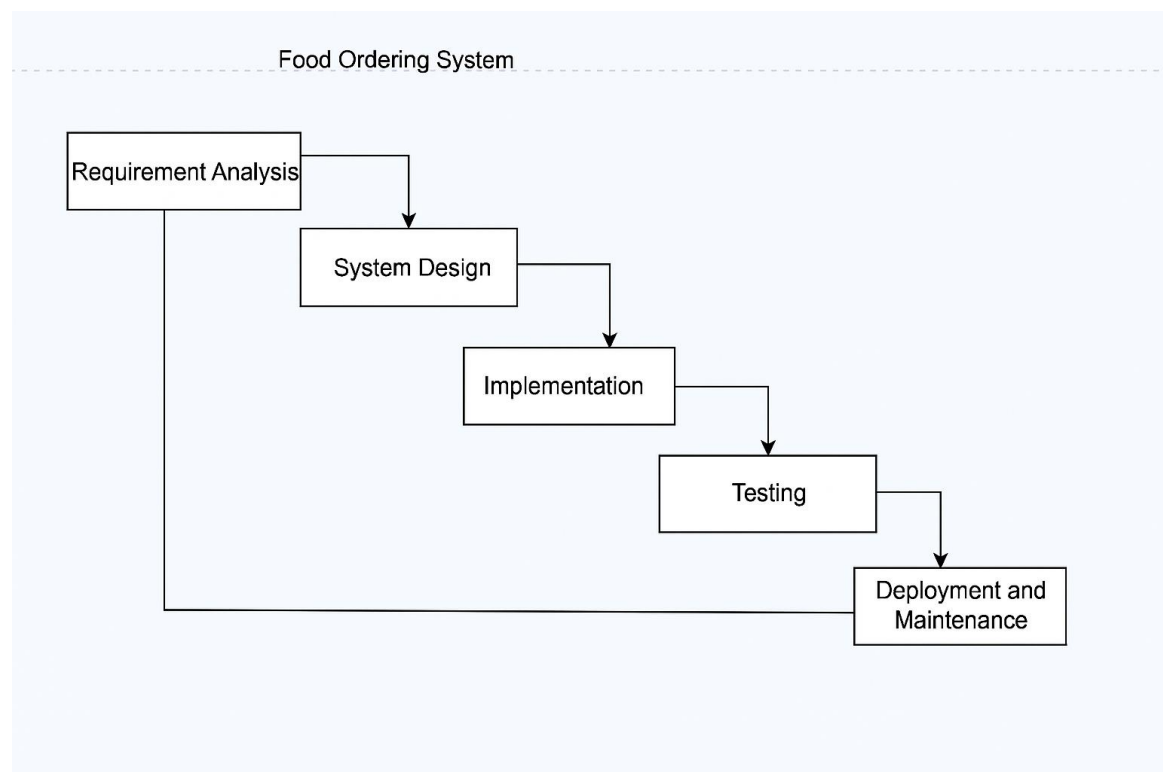


Figure 1: Development Methodology of Food Ordering System

1.6 Report Organization

This report is organized into six chapters, providing a comprehensive overview of the Web-Based Food Ordering System and its development process. Each chapter systematically presents the project's objectives, design, implementation, and evaluation.

1. **Chapter 1 – Introduction:** Presents an overview of the Web-Based Food Ordering System, including the background, problem statement, objectives, scope, limitations, and the adopted Waterfall development methodology.
2. **Chapter 2 – Background Study and Literature Review:** Discusses existing food ordering systems, current technological trends, and related research on web-based platforms, recommendation algorithms, and online service optimization.
3. **Chapter 3 – System Analysis:** Outlines the system's functional and non-functional requirements, feasibility study, and requirement analysis. It also explains how the proposed system addresses the challenges identified in traditional ordering methods.
4. **Chapter 4 – System Design:** Describes the architectural design, database schema, data flow diagrams, and the overall workflow of the system. It also details the design of key components such as the recommendation engine and user interfaces.
5. **Chapter 5 – Implementation and Testing:** Explains the development of the system using the MERN stack, including backend and frontend integration, Stripe payment gateway setup, and the implementation of the Apriori-based recommendation algorithm. This chapter also covers the testing procedures, test cases, and performance analysis.
6. **Chapter 6 – Conclusion and Future Enhancements:** Summarizes the project's achievements, highlights key findings, and discusses limitations faced during development. It also provides recommendations and potential future improvements, such as mobile application support, delivery tracking, and AI-driven analytics.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The rapid growth of information technology has transformed how consumers interact with service industries, particularly in the food and hospitality sector. [5] Traditional food ordering methods such as phone calls or manual orders often lead to inefficiencies, communication errors, and customer dissatisfaction. With the rise of web-based systems, restaurants can now streamline their operations, provide real-time updates, and offer customers a seamless ordering experience.

The Web-Based Food Ordering System aims to address these issues by offering an integrated digital platform where customers can browse menus, place orders, and track their order status online. [6] The system also enables restaurants to efficiently manage menus, orders, and customer data through a centralized interface. [7] The inclusion of an intelligent recommendation engine using the Apriori algorithm further enhances the customer experience by suggesting dishes based on historical order data.

This project is significant because it aligns with the growing trend of digital transformation in the food service industry, especially in urban areas where convenience, speed has become key expectations. The system thus provides both operational efficiency for restaurants and convenience for customers, bridging the communication gap that exists in traditional ordering methods.

2.2 Literature Review

[8] Online food ordering systems have transformed the restaurant industry by offering customers greater convenience, faster service, and personalized experiences. With advancements in web technologies, artificial intelligence, and secure payment systems, these platforms have evolved into intelligent, data-driven applications. The Web-Based Food Ordering System “PetPuja” builds on these developments by integrating a smart recommendation engine using the Apriori algorithm, secure payments through Stripe API, and efficient data handling via a three-tier architecture. This review explores existing

research on online ordering systems, recommendation algorithms, and architectural models to identify current trends and gaps that the PetPuja system addresses.

Modern food ordering platforms streamline operations and improve customer satisfaction through automation and digital interaction. However, many systems still lack real-time adaptive recommendations.[9] Recommendation algorithms like Apriori are widely used to analyze customer purchase history and generate item suggestions based on confidence and support levels. By applying these algorithms, platforms can enhance upselling opportunities and user engagement.

From a design perspective, adopting a three-tier architecture comprising the presentation, application, and data layers ensures system scalability, modularity, and easier maintenance. Frameworks like React.js, Node.js, and Express.js are preferred for building responsive interfaces and high-performance APIs, while MongoDB Atlas provides reliable cloud-based data storage. In addition, secure payment and authentication mechanisms such as JWT-based login and Stripe API integration strengthen data protection and user trust.

Existing systems like Pathao and Foodmandu have contributed significantly to the growth of online food ordering platforms. However, many still operate without real-time recommendations integration between ordering, payment, and notification modules. The PetPuja System overcomes these limitations by combining a recommendation engine, secure payment gateway, and intelligent architecture to deliver a fast, and reliable food ordering experience. This integrated approach make PetPuja as a modern, scalable, and user-centric solution in the digital food service domain.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis involves studying the existing issues in traditional food ordering methods, identifying user and system requirements, and designing an effective digital solution. For the Web-Based Food Ordering System, this phase was vital to ensure that technical and functional components aligned with real-world restaurant operations and customer expectations. Using the Waterfall methodology, requirements were carefully defined and documented before development began. Both functional and non-functional requirements including usability, scalability, and performance were analyzed to create a stable foundation for the system's design and implementation.

3.1.1 Requirement Analysis

Requirement analysis is the process of identifying the essential features and functions the Online Restaurant Management System must have to meet user needs and ensure smooth operation. The analysis includes both functional and non-functional requirements.

Functional Requirements

Functional requirements define what the system must do and the specific features and behaviours it will exhibit. These requirements are directly tied to the user's interaction with the system.

- 1. User Management:** The system allow new users to register, log in, and manage their profile information (e.g., name, contact details, delivery addresses).
- 2. Menu and Order Management:** Customers must be able to browse restaurants, view digital menus, select food items, and add them to a cart. The system must also support order modification and cancellation before the order is confirmed.
- 3. Search and Filter:** The system must provide robust search functionality, allowing users to find restaurants or food items by name, cuisine, location, price range, or dietary tags.
- 4. Payment Integration:** The system must integrate with secure payment gateways to allow for multiple payment options, including digital wallets, credit/debit cards, and cash on delivery.

5. **Recommendation Engine:** Recommendation Engine Powered by the Apriori algorithm, this feature analyzes historic order data to recommend food items.
6. **Order Tracking:** Both customers and restaurants must be able to track the real-time status of an order, from "Order Placed" to "Delivered."
7. **Admin and Restaurant Dashboards:** The system must provide dedicated dashboards for restaurant managers to update menus, manage orders, and view sales reports, and for administrators to oversee the entire platform.



Figure 1: Use Case Diagram of Food Ordering System

Non-Functional Requirements

Non-functional requirements describe the qualities and constraints of the system, defining how well the system performs its functions.

1. **Usability:** The user interface (UI) and user experience (UX) is easy to navigate for all user types, from new customers to restaurant admins.
2. **Reliability:** The system has minimal downtime and be highly available. Error handling should be robust, with clear, helpful messages for users and detailed logs for developers.
3. **Scalability:** The system architecture is designed to scale seamlessly as the number of users, orders grow.
4. **Maintainability:** The codebase is well-structured and documented to allow for easy maintenance, bug fixes, and future feature additions as it uses three-tier architecture.
5. **Security:** The system protect user credentials and payment details through JWT-based authentication, bcrypt password encryption, and OTP Verification.

3.1.2 Feasibility Study

The feasibility study evaluates whether the Online Food Ordering System can be successfully developed, implemented, and maintained. It examines the practicality of the project by analysing the availability of required resources, technology, and infrastructure needed to develop core features such as real-time order tracking, user management, secure payment processing, and menu management. The study also identifies potential challenges that could arise during development or operational phases. This analysis not only determines whether the system can be built but also assesses its ability to function efficiently and deliver value to both restaurant owners and customers.

i. Technical Feasibility

Technical feasibility assesses whether the Web-Based Food Ordering System can be successfully implemented based on available resources and technology. The project is technically feasible as it leverages widely-used and supported technologies, including MERN stack (MongoDB, Express.js, React.js, Node.js) for frontend development and Backend and real-time communication. These technologies are robust, scalable, and have extensive documentation and community support, ensuring that the development team can access the resources necessary for successful implementation.

ii. Operational Feasibility

The system's design will support a high degree of operational flexibility for all key stakeholders:

For Admin: The Admin Dashboards will provide a robust set of tools for menu management, order fulfilment, and analytics, reducing the manual effort required for order taking and tracking.

For Customers: The user-friendly interface and tracking will provide a seamless ordering experience, from browsing to delivery. The system's ability to handle dynamic prices and promotions allows for agile marketing campaigns.

iii. Economic Feasibility

The chosen technology stack is economically feasible and helps to reduce both development and operational costs. The use of open-source frameworks like React, Node.js, and Express.js, along with a free-to-use database like MongoDB, minimizes licensing fees. The system's efficiency in managing orders and deliveries reduces the need for extensive manual labour, and its scalability ensures that the initial investment can support future business growth without a significant overhaul. The integration with a scalable payment gateway like Stripe further ensures cost-effective transaction processing.

iv. Schedule Feasibility

Schedule feasibility assesses whether the project can be completed within the given timeframe. The development of the Web-Based Food Ordering System is feasible within the specified schedule. The project timeline will be managed using tools like Gantt charts to outline milestones, tasks, and deadlines. With a dedicated development team and clear project phases, the system can be developed, tested, and deployed within the agreed-upon timeframe. The planned schedule allows for sufficient time for each stage of the project, including planning, development, testing, and deployment.

SN	Task	Start	Duration	End
1	Project Planning	2025-07-15	7	2025-07-22
2	Requirement Gathering & Analysis	2025-07-22	17	2025-08-08
3	Data Collection	2025-08-08	12	2025-08-20
4	System Design (UI & DB)	2025-08-20	14	2025-09-03
5	Implementation (Frontend & Backend Dev)	2025-09-03	24	2025-09-27
6	Algorithm Development & Integration	2025-09-27	17	2025-10-14
7	Integration & Testing	2025-10-14	20	2025-11-03
8	Final Testing & Bug Fixing	2025-11-03	10	2025-11-13
9	Documentation & Report Preparation	2025-07-15	119	2025-11-15

Table 1: Estimated Project Schedule

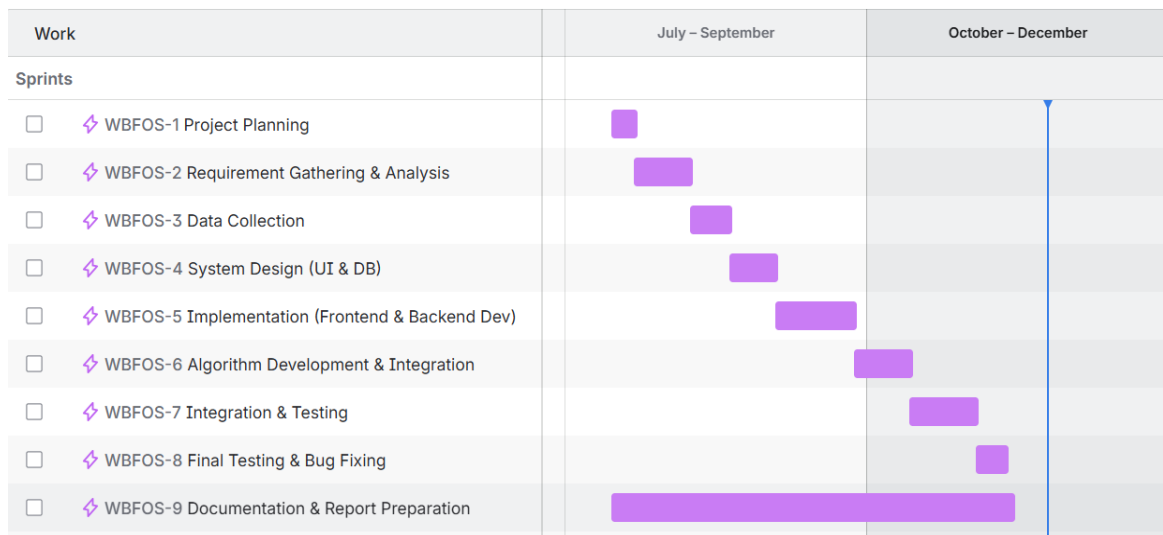


Figure 2: Gantt chart of Project schedule

3.2 Analysis

A class diagram represents a system's classes, interfaces, and relationships. It shows the static structure of the system and models the classes, attributes, methods, and their relationships.

3.2.1 Object modelling using Class Diagram:

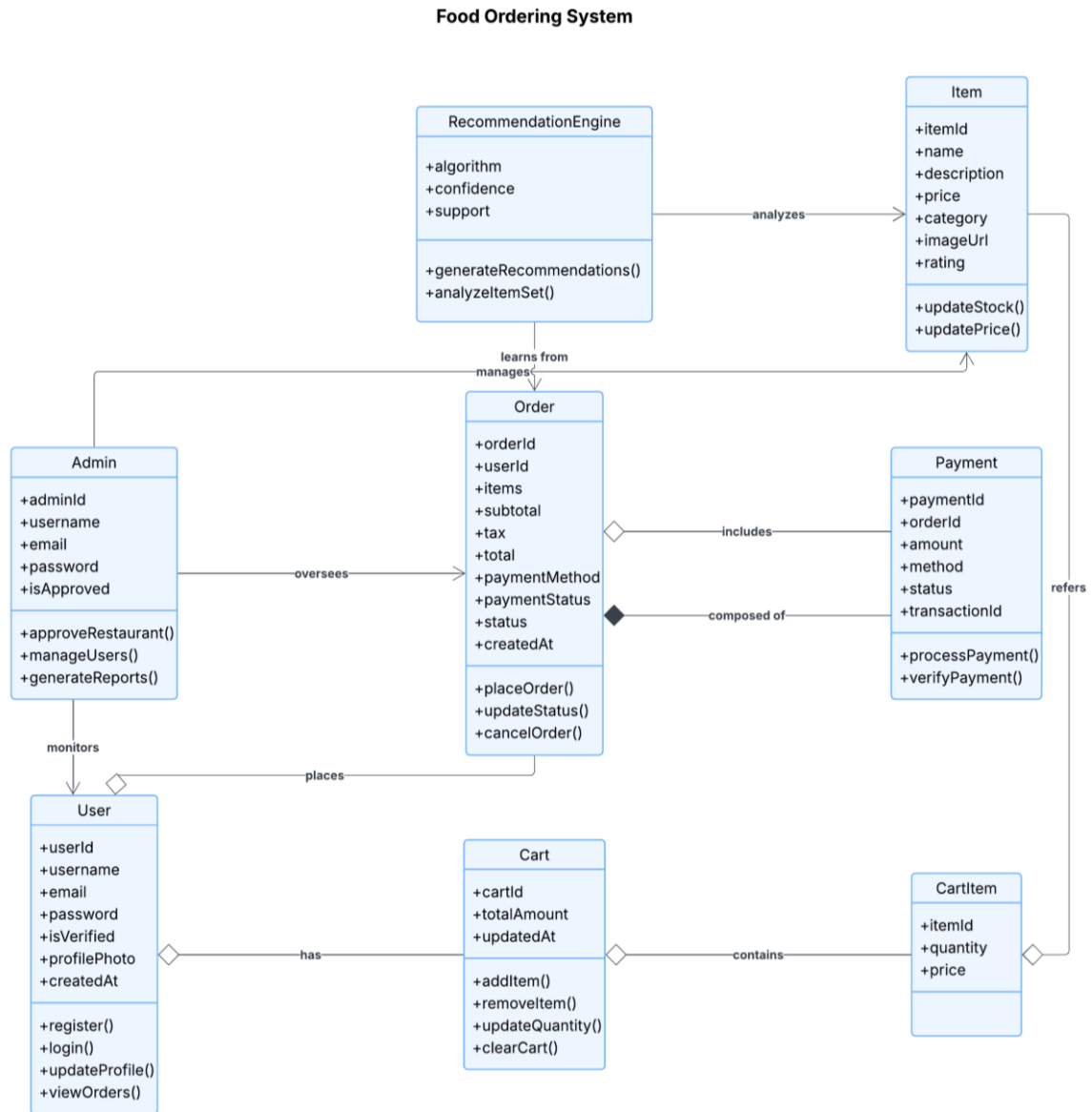


Figure 3 Class Diagram of Food Ordering System

The Web-Based Food Ordering System class diagram illustrates the structure and behavior of the system through the interaction of its main components. There are eight principal classes in the system: Admin, User, Cart, CartItem, Item, Order, Payment, and RecommendationEngine. The Admin class manages administrative operations such as approving restaurants with approveRestaurant(), managing users through manageUsers(), and generating reports using generateReports(). The User class performs customer-related tasks, including registration register(), login login(), profile updates updateProfile(), and

viewing orders `viewOrders()`. The `Cart` and `CartItem` classes handle shopping cart management with methods like `addItem()`, `removeItem()`, `updateQuantity()`, and `clearCart()`. The `Item` class maintains details of each menu item through `updateStock()` and `updatePrice()`, while the `Order` class coordinates the ordering process with `placeOrder()`, `updateStatus()`, and `cancelOrder()`. The `Payment` class ensures secure transaction handling via `processPayment()` and `verifyPayment()`, and the `RecommendationEngine` class enhances the customer experience by analyzing purchase data through `generateRecommendations()` and `analyzeItemSet()` to provide intelligent food suggestions.

3.2.2 Dynamic modelling using State Diagram

The Web-Based Food Ordering System state diagram illustrates the complete process flow from user authentication to order completion. The operation begins when the system is initialized, and the user attempts to log in by entering credentials. If the credentials are valid, the user gains access to the system; otherwise, access is denied, prompting the user to re-enter the credentials or sign up for a new account. Upon successful authentication, the user can browse menu items, add food to the cart, view the cart, and receive personalized recommendations. Once satisfied, the user proceeds to place an order, and the system confirms the order and allows the user to view its current status. The next phase involves payment processing; if the payment is successful, the order is confirmed and delivered to the user; if the payment fails, the user is prompted to retry or select another payment method. After the successful completion of the order, the user logs out, and the system returns to the idle state.

This state diagram effectively demonstrates the dynamic behaviour and logical transitions within the Web-Based Food Ordering System.

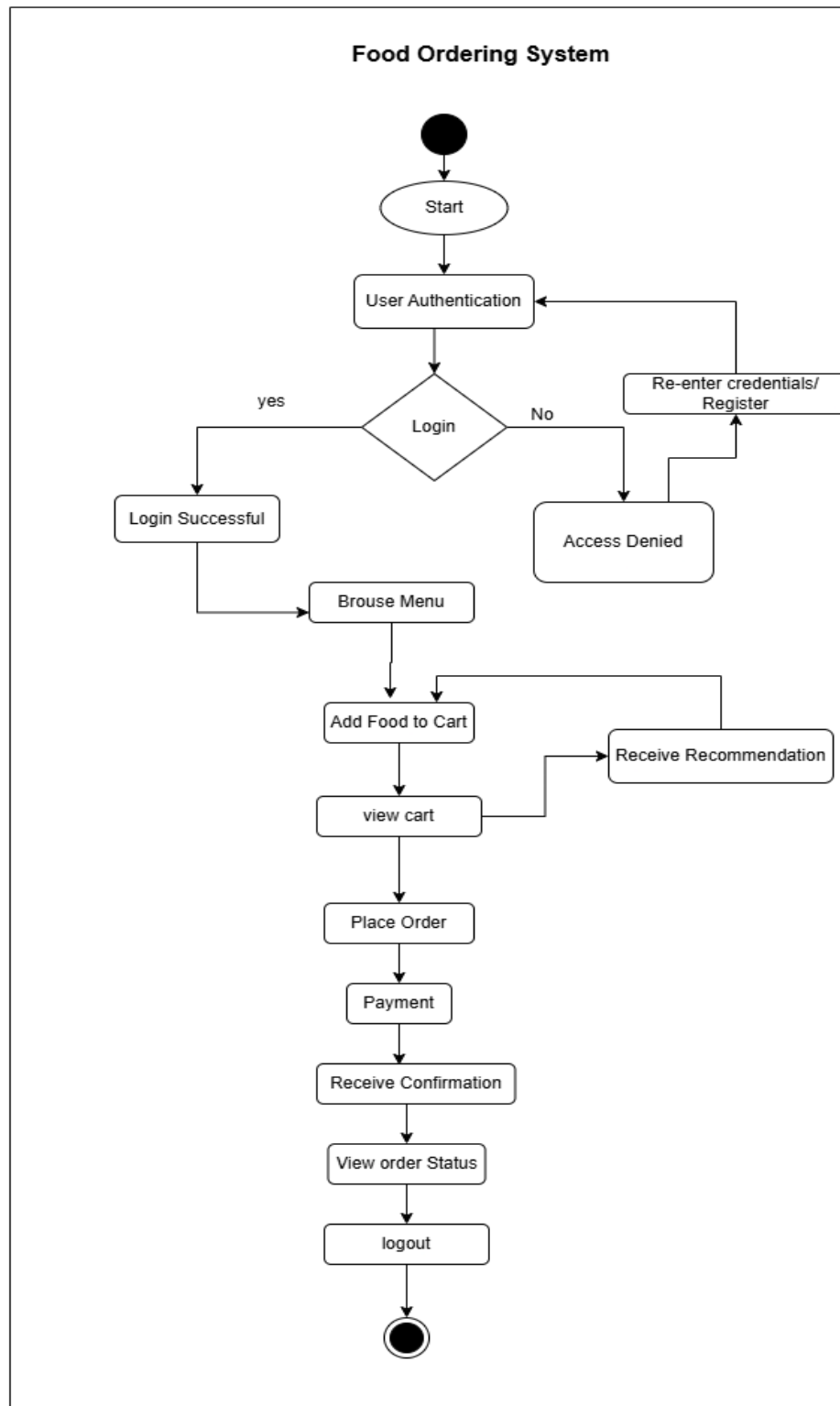


Figure 4 State Diagram of Food Ordering System

3.2.3 Dynamic modeling using Sequence Diagram

The Sequence Diagram in UML represents how different objects interact with each other and the order in which these interactions occur. It focuses on illustrating the flow of communication between components in a particular scenario. Processes are arranged vertically, while the interactions between them are shown as arrows to indicate the sequence of operations. The Web-Based Food Ordering System sequence diagram show the interaction between the User, System, Database, and Model.

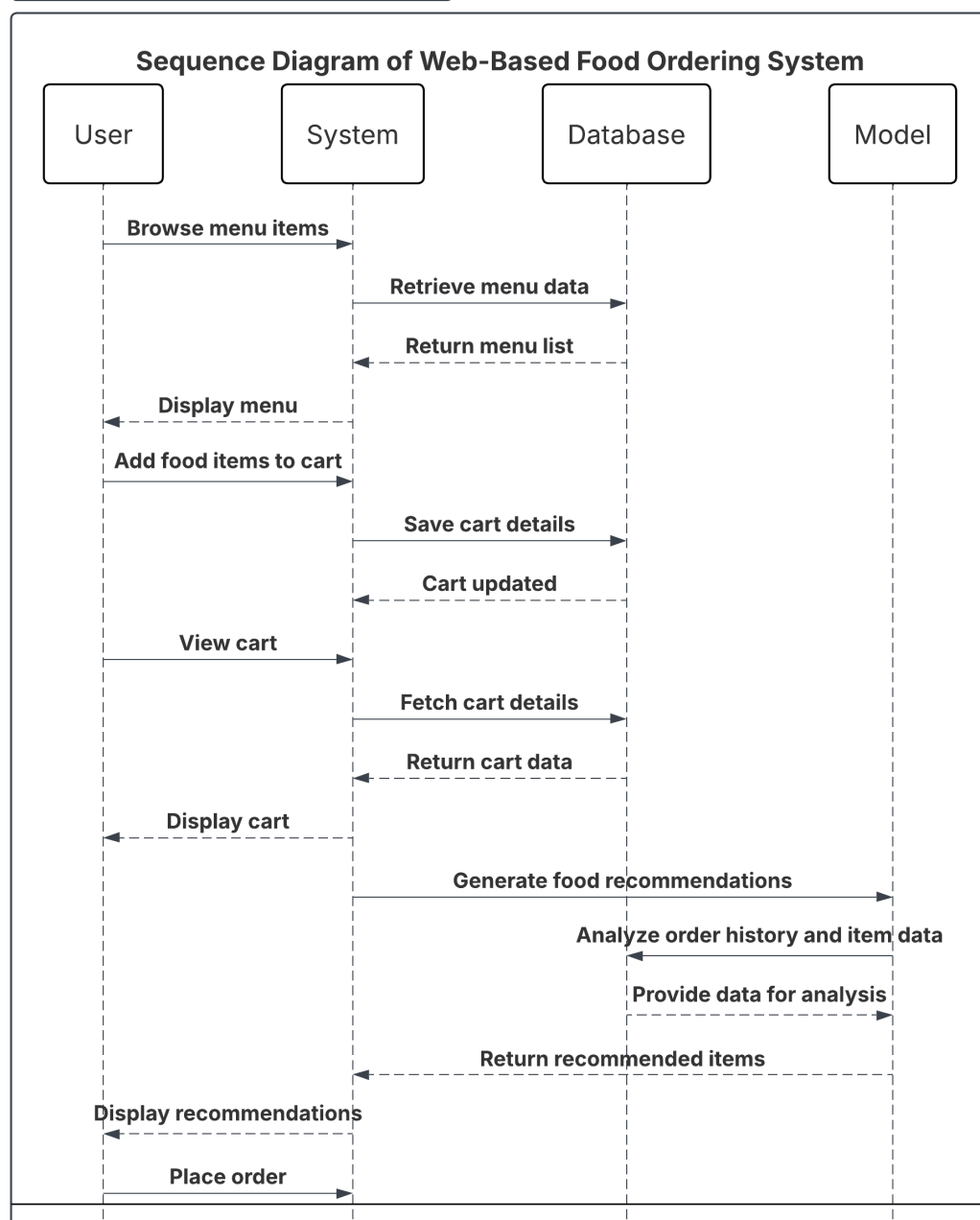


Figure 5 Sequence Diagram of Food Ordering System

The flow begins when the user browses the menu and selects food items, which the system retrieves from the database. The user adds items to the cart and views it, after which the system interacts with the model to generate food recommendations based on order history and item data. Once the user places the order, the system saves the details in the database and confirms the order. Finally, the user proceeds to make the payment, and the system updates the payment status in the database, confirming that the order has been successfully processed.

3.2.4 Process modelling using Activity Diagrams

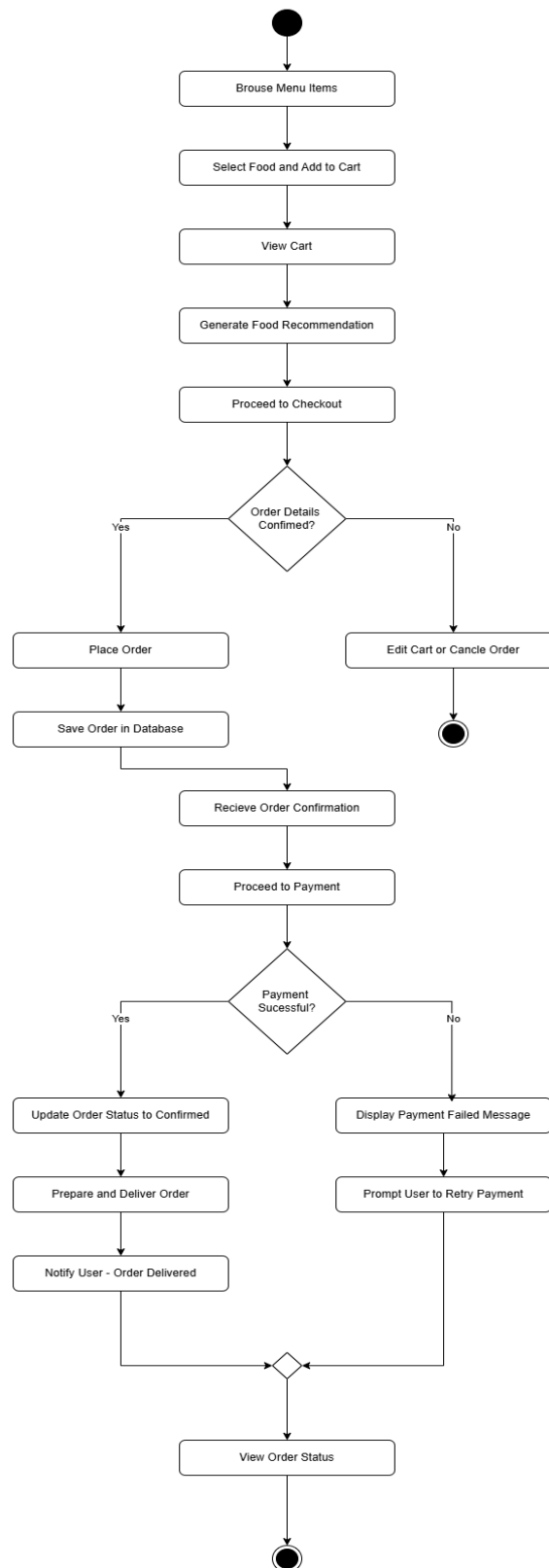


Figure 6: Activity Diagram of Food Ordering System

An Activity Diagram for a Web-Based Food Ordering System represents the sequence of activities involved in browsing, ordering, and completing a food transaction. It provides a step-by-step visualization of how the user interacts with the system to achieve the desired outcome. The activity diagram illustrates the overall process flow of the system. The process begins when the user browses the available menu items and adds selected food items to the cart. The user can then view the cart and receive personalized food recommendations generated by the system. After reviewing the order, the user proceeds to checkout, where the order details are verified and saved in the database. Once the order is confirmed, the user initiates the payment process. If the payment is successful, the system updates the order status, initiates food preparation, and marks the order as delivered. If the payment fails, the user is prompted to retry the transaction. This systematic flow ensures a smooth and efficient process from menu selection to final order completion within the Web-Based Food Ordering System.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

Following the study and design of the system, implementation is the next step. The code is required to produce the desired result. The verification and validation of the coding exist to determine whether all of the generated systems meet the user requirements. The various tools aid in building modules, which provide a starting point for coding. In addition, testing must end before the system launch.

4.1.1 Analysis and Design Tools

- 1) **Draw.io/Lucidchart** :Draw.io is an diagramming tool used for creating professional system and software design diagrams. It supports various UML models such as use-case diagrams, activity diagram, system flowchart etc. Where as lucid is another diagramming tool which was used to make class diagram and sequence diagram.
- 2) **Jira** :In this project, Jira was used to develop the Gantt chart and schedule for the project tasks. Jira allowed the project activities to be broken down into smaller, manageable tasks, with each task assigned start and end dates. Using Jira's project timeline view, the tasks were arranged in sequence to show dependencies, duration, and overall workflow. This helped in visualising the progress of the project, identifying critical deadlines, and ensuring that each phase of development was completed on time.
- 3) **MS Word (2019)** :Microsoft Word was used for preparing the project documentation and report writing, including requirement analysis, design specifications, testing results, and implementation details. It provided advanced formatting, referencing, and collaboration features, ensuring a professional, well-structured, and consistent project report.
- 4) **VS Code**: Visual Studio Code was used as the primary code editor in this project. It provided a simple and user-friendly interface, along with helpful features such as syntax highlighting, code suggestions, and debugging tools. The built-in terminal and support for extensions made it easy to run the project and integrate tools like Git for version control. Overall, VS Code supported efficient coding, testing, and project management throughout development.

4.1.2 Tools and Technologies

1. Programming Languages & Frameworks

Node.js: Used as the backend runtime environment to manage server-side logic, request handling, and API communication.

Express.js: A lightweight Node.js framework used to build RESTful APIs for user authentication, product management, cart operations, and order processing.

React with Tailwind CSS: Used on the frontend to build a responsive, component-based user interface. Tailwind CSS enabled fast styling, while React supported dynamic UI rendering and state management.

Framer Motion: Integrated for smooth, modern UI animations and transitions, enhancing the overall user experience.

MongoDB with Mongoose: MongoDB served as the NoSQL database to store users, carts, products, and orders. Mongoose provided schema modelling and validation for consistent data handling.

Stripe API: Implemented for secure and real-time online payment processing.

2. Database and Hosting

MongoDB Atlas: Used to host and manage the database in the cloud, providing data backup, security, and remote accessibility.

3. Version Control & Collaboration

Git: Used locally for version control to track changes, manage branches, and maintain clean development workflow.

GitHub: Served as the remote repository for collaboration, pull requests, and safe cloud backup of the source code.

4. Accounts and External Services

MongoDB Atlas Account: For managing and connecting the cloud-hosted database.

Stripe Account: For enabling and managing payment transactions through the Stripe API.

Gmail Account: Used for email verification, notifications, and communication services.

4.2 Implementation Details of Modules

i. Apriori Algorithm module

a. trainFetch Module

Code Snipped:

```
export const trainRecommendationModel = async (req, res) => {  
  try {  
    // Fetch all completed orders  
    const orders = await Order.find({ status: 'delivered' });  
  
    if (orders.length === 0) {  
      return res.json({  
        success: false,  
        message: 'No order history available for training'  
      });  
    }  
  }  
}
```

This module trains the recommendation engine of the TrioOrder System using past delivered orders. It retrieves all completed orders from the database and converts them into transaction data for the Apriori algorithm. The algorithm identifies frequently bought-together items, which are then used to generate food recommendations. If no past orders are found, the system returns a message indicating insufficient data for training.

b. Itemset Generation Module

Code Snipped:

```
class AprioriAlgorithm {  
  constructor(minSupport = 0.1, minConfidence = 0.5) {  
    this.minSupport = minSupport;  
    this.minConfidence = minConfidence;  
  }  
}
```

```

}

/**
 * Generate all itemsets of size k from itemsets of size k-1
 */
generateCandidates(prevItemsets, k) {
  const candidates = [];
  const len = prevItemsets.length;

  for (let i = 0; i < len; i++) {
    for (let j = i + 1; j < len; j++) {
      const itemset1 = prevItemsets[i];
      const itemset2 = prevItemsets[j];

      // Join step: combine itemsets if they differ by only one item
      const union = [...new Set([...itemset1, ...itemset2])];
      if (union.length === k) {
        // Sort to avoid duplicates
        const sortedUnion = union.sort();
        const exists = candidates.some(candidate =>
          JSON.stringify(candidate) === JSON.stringify(sortedUnion)
        );
        if (!exists) {
          candidates.push(sortedUnion);
        }
      }
    }
  }

  return candidates;
}

```

The AprioriAlgorithm class helps the system find patterns in customer orders by identifying which food items are frequently bought together. This is a key part of the recommendation feature in the TrioOrder system. The class uses two important parameters:

- Minimum Support (0.1 or 10%) – determines how often a combination of items must appear in past orders to be considered important.
- Minimum Confidence (0.5 or 50%) – measures how reliable the relationship between items is when creating a recommendation rule.

The function generateCandidates(prevItemsets, k) generates new possible item combinations (called candidate itemsets) of size k from previously found smaller sets. It does this by joining pairs of itemsets that differ by only one item and removing any duplicates. These generated combinations are then analyzed to check which ones meet the support and confidence value.

c. Calculate Support and Prune Module

Code Snipped:

```
* Calculate support count for an itemset
*/
calculateSupport(itemset, transactions) {
  let count = 0;
  for (const transaction of transactions) {
    if (itemset.every(item => transaction.includes(item))) {
      count++;
    }
  }
  return count / transactions.length;
}

/**
 * Prune candidates based on minimum support
 */
pruneCandidates(candidates, transactions, frequentItemsets) {
  for (const candidate of candidates) {
```

```

const support = this.calculateSupport(candidate, transactions);
if (support >= this.minSupport) {
  frequentItemsets.push({
    itemset: candidate,
    support: support
  });
}
}
return frequentItemsets;
}

```

The `calculateSupport(itemset, transactions)` function measures how often a specific combination of items (called an itemset) appears across all past orders. It counts how many transactions include every item in that set, then divides that count by the total number of transactions to get a support value — essentially showing how popular that combination is among customers.

The `pruneCandidates(candidates, transactions, frequentItemsets)` function uses the calculated support values to filter out itemsets that don't meet the minimum support threshold (in this case, 0.1 or 10%). Only the combinations that occur frequently enough are kept and added to the list of frequent itemsets.

d. Frequent Itemset Module

Code Snipped:

```

// Generate frequent 1-itemsets
const itemCounts = {};
for (const transaction of transactions) {
  for (const item of transaction) {
    itemCounts[item] = (itemCounts[item] || 0) + 1;
  }
}

```

```

let currentItemsets = [];
for (const [item, count] of Object.entries(itemCounts)) {
  const support = count / transactions.length;
  if (support >= this.minSupport) {
    currentItemsets.push({
      itemset: [item],
      support: support
    });
  }
}

allFrequentItemsets.push(...currentItemsets);

// Generate frequent k-itemsets
let k = 2;
while (currentItemsets.length > 0) {
  const prevItemsets = currentItemsets.map(item => item.itemset);
  const candidates = this.generateCandidates(prevItemsets, k);

  currentItemsets = [];
  this.pruneCandidates(candidates, transactions, currentItemsets);

  if (currentItemsets.length > 0) {
    allFrequentItemsets.push(...currentItemsets);
  }

  k++;
}

return allFrequentItemsets;
}

```

The `findFrequentItemsets()` function is the core part of the Apriori algorithm. It identifies all combinations of items that frequently appear together in customer orders, which later helps the system make accurate food recommendations.

The process begins by analyzing all transactions (past delivered orders) to count how often each individual item appears. These single items are checked against the minimum support threshold (for example, 0.1 or 10%) only items that meet or exceed this value are kept as frequent 1-itemsets.

Next, the algorithm gradually builds larger item combinations (2-itemsets, 3-itemsets, and so on) by joining previously found frequent sets and checking their support again. This process continues until no new frequent itemsets can be found. The final result is a complete list of all item combinations that occur often enough to be considered significant.

e. Rule Generation Module

Code Snipped:

```
generateRules(frequentItemsets) {  
  const rules = [];  
  
  // Only consider itemsets with 2 or more items  
  const multiItemsets = frequentItemsets.filter(item => item.itemset.length >= 2);  
  
  for (const itemsetObj of multiItemsets) {  
    const itemset = itemsetObj.itemset;  
    const itemsetSupport = itemsetObj.support;  
  
    // Generate all possible subsets as antecedents  
    const subsets = this.generateSubsets(itemset);  
  
    for (const antecedent of subsets) {  
      if (antecedent.length === 0 || antecedent.length === itemset.length) {  
        continue;  
      }  
    }  
  }  
}
```

```

const consequent = itemset.filter(item => !antecedent.includes(item));

// Find support of antecedent
const antecedentSupport = frequentItemsets.find(item =>
  item.itemset.length === antecedent.length &&
  antecedent.every(a => item.itemset.includes(a))
)?.support || 0;

if (antecedentSupport > 0) {
  const confidence = itemsetSupport / antecedentSupport;

  if (confidence >= this.minConfidence) {
    rules.push({
      antecedent: antecedent,
      consequent: consequent,
      support: itemsetSupport,
      confidence: confidence,
    });
  }
}

// Sort rules by confidence (descending)
return rules.sort((a, b) => b.confidence - a.confidence);
}

/**
 * Calculate support for consequent
 */
calculateConsequentSupport(consequent, frequentItemsets) {
  const found = frequentItemsets.find(item =>
    item.itemset.length === consequent.length &&

```

```

    consequent.every(c => item.itemset.includes(c))
  );
  return found ? found.support : 0.01; // Avoid division by zero
}

/**
 * Generate all subsets of an array
 */
generateSubsets(arr) {
  const subsets = [[]];

  for (const item of arr) {
    const len = subsets.length;
    for (let i = 0; i < len; i++) {
      subsets.push([...subsets[i], item]);
    }
  }

  return subsets;
}

```

The Rule Generation Module is responsible for creating association rules from the frequent itemsets discovered by the Apriori algorithm. Its main function, `generateRules(frequentItemsets)`, examines all frequent combinations of food items that customers commonly order together and generates logical “if-then” relationships.

For example, if customers frequently order Pizza and Coke together, the module produces a rule such as:

If a customer orders Pizza, they are likely to also order Coke.

The function works by first identifying all itemsets that contain at least two items. It then generates subsets (called antecedents and consequents) and calculates two key measures for each rule:

- Support – how often the entire itemset appears in past transactions.
- Confidence – how reliably the presence of one item predicts the occurrence of another, based on the minimum confidence threshold (e.g., 0.5 or 50%).

The generated rules are then sorted by confidence, ensuring that the most reliable and meaningful recommendations are prioritized.

f. Recommendation Generation Module

Code Snipped:

```
getRecommendations(cartItems, rules, limit = 5) {
  const recommendations = new Map();

  // Find rules where antecedent matches cart items
  for (const rule of rules) {
    // Check if all antecedent items are in cart
    const matchingItems = rule.antecedent.filter(item => cartItems.includes(item));

    if (matchingItems.length > 0) {
      // Add consequent items as recommendations
      for (const item of rule.consequent) {
        // Don't recommend items already in cart
        if (!cartItems.includes(item)) {
          if (!recommendations.has(item)) {
            recommendations.set(item, {
              itemId: item,
              score: rule.confidence,
              support: rule.support,
              basedOn: rule.antecedent
            });
          } else {
            // If item already recommended, take highest confidence
            const existing = recommendations.get(item);
```

```

        if (rule.confidence > existing.score) {
            recommendations.set(item, {
                itemId: item,
                score: rule.confidence,
                support: rule.support,
                basedOn: rule.antecedent
            });
        }
    }
}

// Convert to array and sort by score
const sortedRecommendations = Array.from(recommendations.values())
    .sort((a, b) => b.score - a.score)
    .slice(0, limit);

return sortedRecommendations;
}

```

The Recommendation Generation Module is responsible for providing personalized food suggestions to users based on the items currently in their shopping cart. It uses the association rules generated by the Apriori algorithm to identify items that are frequently ordered together.

The function `getRecommendations(cartItems, rules, limit = 5)` checks each rule to find cases where the antecedent (the “if” part of the rule) matches one or more items in the customer’s cart. When a match is found, the consequent (the “then” part) of that rule is added as a potential recommendation unless the item is already in the cart. For each recommended item, the system records key values such as confidence, support, and the items it’s based on.

If the same item is suggested by multiple rules, the system keeps the recommendation with the highest confidence score to ensure quality suggestions. Finally, all recommendations are sorted in descending order of confidence, and only the top five results are returned.

ii. Recommendation Retrieval and Fallback Module

Code Snipped:

```
/**
 * Get popular items as fallback
 */
const getPopularItems = async () => {
  try {
    // Get items sorted by popularity (hearts) and rating
    const items = await Item.find({})
      .sort({ hearts: -1, rating: -1 })
      .limit(10);
    return items;
  } catch (error) {
    console.error('Error fetching popular items:', error);
    return [];
  }
};
```

The Recommendation Retrieval and Fallback Module is responsible for fetching the complete details of the food items identified by the recommendation engine and preparing them for display to the user. After the system generates a list of recommended items using the Apriori algorithm, this module retrieves each item's full information such as its name, description, price, and image from the database using the corresponding item IDs.

Once the item details are fetched, the module combines them with their respective recommendation scores and the list of base items that influenced each suggestion. This results in a set of enriched recommendations that include both item data and performance metrics, which are then sent to the frontend for the customer to view.

Additionally, the module includes a fallback mechanism (getPopularItems) that activates when there isn't enough historical order data to train the recommendation model. In such cases, it fetches and displays the most popular menu items determined by their hearts (likes) and ratings ensuring that users still receive relevant suggestions even when recommendation data is limited

iii. Admin Registration Module

This module handles the secure registration of admin accounts. When an admin signs up, the system first checks if the email already exists. The password is validated and then hashed using bcrypt for security. Unlike normal users, newly registered admins are not activated immediately. Instead, the system generates an approval token and sends a request to the system owner to approve the account. Only after the owner approves and the admin verifies their email, the admin account becomes active and allowed to log in. This ensures that only authorized personnel gain administrative access and prevents unauthorized privilege escalation.

Code Snipped:

```
import adminModel from "../models/adminModel.js";
import bcrypt from "bcrypt";
import validator from "validator";
import crypto from "crypto";
import { sendOwnerApprovalRequest } from "../utils/emailService.js";

const OWNER_EMAIL = "triotrick0630@gmail.com";

// Admin Register Module
const registerAdmin = async (req, res) => {
  const { username, password, email } = req.body;

  try {
    // Check if admin already exists
    const exists = await adminModel.findOne({ email });

    if (exists) {
```

```

// Already registered but waiting for approval
if (!exists.isApprovedByOwner) {
  return res.json({
    success: false,
    message: "Your registration is pending owner approval. Please wait for
confirmation email.",
    pendingApproval: true
  });
}

// Approved but email not verified
if (exists.isApprovedByOwner && !exists.isVerified) {
  return res.json({
    success: false,
    message: "Your account was approved. Please check your email for verification
PIN.",
    needsVerification: true,
    email: email
  });
}

return res.json({ success: false, message: "Admin already exists and is verified.
Please login." });
}

// Validate email format
if (!validator.isEmail(email)) {
  return res.json({ success: false, message: "Please enter a valid email" });
}

// Validate password strength
if (password.length < 8) {
  return res.json({

```

```

        success: false,
        message: "Password must be at least 8 characters long",
    });
}

// Hash password
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);

// Generate owner approval token
const approvalToken = crypto.randomBytes(32).toString('hex');

// Save admin as pending approval
const newAdmin = new adminModel({
    username,
    email,
    password: hashedPassword,
    isApprovedByOwner: false,
    approvalToken,
    isVerified: false
});

const admin = await newAdmin.save();
console.log("Admin registered (pending owner approval):", admin._id);

// Send approval request to owner
await sendOwnerApprovalRequest(OWNER_EMAIL, { username, email },
approvalToken);

res.json({
    success: true,
    message: "Registration request submitted. Await owner approval.",
    pendingApproval: true
});

```

```

});

} catch (error) {
  console.error("✖ Admin registration error:", error);
  res.status(500).json({ success: false, message: "Admin registration failed" });
}
};

export default registerAdmin;

```

iv. Admin Login Module

Code Snipped:

```

// Admin Login Module
const createAdminToken = (id, email) => {
  return jwt.sign({ id, email, role: 'admin' }, process.env.JWT_SECRET);
};

const loginAdmin = async (req, res) => {
  const { email, password } = req.body;

  try {
    // Check if admin exists
    const admin = await adminModel.findOne({ email });
    if (!admin) {
      return res.json({ success: false, message: "Admin account doesn't exist" });
    }

    // Check if owner has approved the admin
    if (!admin.isApprovedByOwner) {
      return res.json({
        success: false,

```

```

        message: "Your registration is pending owner approval. Please wait for
confirmation email.",
        pendingApproval: true
    });
}

// Check email verification status
if (!admin.isVerified) {
    return res.json({
        success: false,
        message: "Please verify your email first. Check your inbox for verification PIN.",
        needsVerification: true,
        email: admin.email
    });
}

// Validate password
const isMatch = await bcrypt.compare(password, admin.password);
if (!isMatch) {
    return res.json({ success: false, message: "Invalid password" });
}

// Update last login timestamp
admin.lastLogin = new Date();
await admin.save();

// Generate admin JWT token
const token = createAdminToken(admin._id, admin.email);

res.json({
    success: true,
    token,
    admin: {

```

```

        id: admin._id,
        username: admin.username,
        email: admin.email,
        role: admin.role
      }
    });

  } catch (error) {
    console.error('Admin login error:', error);
    res.json({ success: false, message: "Login failed" });
  }
};

```

The Admin Login module is responsible for securely authenticating administrators before granting system access. When an admin attempts to log in, the module first checks if the admin account exists in the database. It then verifies whether the account has been approved by the system owner and whether the email has been successfully verified. After this validation, the entered password is compared with the stored hashed password using bcrypt. If the credentials are valid, the system generates a JWT token containing the admin's ID and role, which is returned to the client for authenticated access. This module ensures that only verified and owner-approved administrators can access the system.

v. Admin Authentication Module

This module ensures **secure access control for administrative functions** in the system. It uses **JWT-based role verification** to restrict sensitive operations exclusively to authorized admin users.'

Code Snippet:

```

import jwt from 'jsonwebtoken';
const adminAuthMiddleware = (req, res, next) => {
  console.log('Admin auth middleware called for:', req.path);
  const token =

```

```

req.cookies?.token ||
(req.headers.authorization && req.headers.authorization.split(' ')[1]);

console.log('Token found:', !!token);

if (!token) {
  console.log('No token found, returning 401');
  return res.status(401).json({ success: false, message: 'Admin token missing' });
}

try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  console.log('Token decoded successfully for user:', decoded.email);

  // Check if this is an admin token (has role: 'admin')
  if (decoded.role !== 'admin') {
    console.log('Token is not an admin token, role:', decoded.role);
    return res.status(403).json({ success: false, message: 'Admin access required' });
  }

  req.admin = { _id: decoded.id, email: decoded.email };
  next();
} catch (err) {
  console.error('Token verification failed:', err.message);
  const message =
    err.name === 'TokenExpiredError' ? 'Admin token expired' : 'Invalid admin token';
  res.status(403).json({ success: false, message });
}
};

export default adminAuthMiddleware;

```

Token Validation

The module checks for a valid JWT token in either the request cookies or the Authorization header. The token is then verified using `jwt.verify()` along with the secret key stored in the `.env` file. If the token is missing or invalid, access is immediately denied.

Admin Role Verification

After decoding the token, the module checks whether the role field is set to "admin". Only tokens issued to admins contain this role. If the token belongs to a normal user or the role is missing, the request is blocked with a **403 – Forbidden** response.

Request Forwarding

If the token is valid and the user has the admin role, the decoded admin details (ID and email) are attached to `req.admin` and the request is forwarded to the controller. This ensures that only authenticated administrators can perform privileged actions such as managing menus, orders, offers, and system configurations.

Security Guarantee

This module prevents unauthorized access to backend administrative routes and ensures the system remains protected from privilege escalation attacks. Only verified and approved administrators can access restricted functionality.

vi. User Register Module

Code Snipped:

```
// Create JWT Token
const createToken = (id, email) => {
  return jwt.sign({ id, email }, process.env.JWT_SECRET);
};

// Register User
export const registerUser = async (req, res) => {
  const { username, password, email } = req.body;

  try {
    // Check if user already exists
    const exists = await userModel.findOne({ email });
```

```

// If exists but not verified, resend verification PIN
if (exists && !exists.isVerified) {
  const pin = generatePin();
  exists.verificationPin = pin;
  exists.verificationPinExpires = new Date(Date.now() + 15 * 60 * 1000);
  await exists.save();
  await sendVerificationEmail(email, username, pin);

  return res.json({
    success: false,
    message: "Account exists but not verified. New PIN sent to email.",
    needsVerification: true,
    email: email
  });
}

if (exists) {
  return res.json({ success: false, message: "User already exists. Please login." });
}

// Validate Email Format
if (!validator.isEmail(email)) {
  return res.json({ success: false, message: "Invalid email address" });
}

// Validate Password Length
if (password.length < 8) {
  return res.json({ success: false, message: "Password must be at least 8 characters
long" });
}

// Hash Password

```

```

const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);

// Create Verification PIN
const pin = generatePin();
const pinExpires = new Date(Date.now() + 15 * 60 * 1000);

// Save New User (Unverified)
const newUser = new userModel({
  username,
  email,
  password: hashedPassword,
  isVerified: false,
  verificationPin: pin,
  verificationPinExpires: pinExpires
});

const user = await newUser.save();

// Send Verification Email
await sendVerificationEmail(email, username, pin);

res.json({
  success: true,
  message: "Registration successful! Check email for verification PIN.",
  needsVerification: true,
  email: email
});

} catch (error) {
  console.error("User Registration Error:", error);
  res.status(500).json({ success: false, message: "Registration failed" });
}

```

```
};
```

The User Registration Module is responsible for securely creating new user accounts in the system. When a user submits their username, email, and password, the module first checks whether the account already exists. It validates the email format and enforces a secure password policy. The password is then hashed using bcrypt to prevent storing plain-text credentials.

A verification PIN is generated and sent to the user's email to ensure the email address is valid. The user account is stored in the database in an unverified state until the PIN is confirmed. If the user had previously registered but not verified their email, the module automatically resends a new verification PIN.

This module ensures that only verified and authenticated users can access the platform, enhancing security, authenticity, and user identity validation within the system.

vii. User Login Module

Code Snipped:

```
// Generate JWT Token
const createToken = (id, email) => {
  return jwt.sign({ id, email }, process.env.JWT_SECRET);
};

// User Login Module
const loginUser = async (req, res) => {
  const { email, password } = req.body;

  try {
    // Check if user exists
    const user = await userModel.findOne({ email });
    if (!user) {
      return res.json({ success: false, message: "User doesn't exist" });
    }
  }
```

```
// Ensure email verification
if (!user.isVerified) {
  return res.json({
    success: false,
    message: "Please verify your email first.",
    needsVerification: true,
    email: user.email
  });
}

// Check if account is active
if (!user.isActive) {
  return res.json({
    success: false,
    message: "Your account is deactivated. Please contact administrator.",
    accountDeactivated: true
  });
}

// Verify password using bcrypt
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) {
  return res.json({ success: false, message: "Invalid password" });
}

// Update last active status
user.lastActive = new Date();
user.isActive = true;
await user.save();

// Generate JWT token
const token = createToken(user._id, user.email);
```

```

res.json({
  success: true,
  token,
  user: {
    id: user._id,
    username: user.username,
    email: user.email,
    profilePhoto: user.profilePhoto
  }
});

} catch (error) {
  console.error('Login error:', error);
  res.json({ success: false, message: "Login failed" });
}
};

export { loginUser };

```

The User Login Module is responsible for authenticating users when they attempt to access the system. It first checks whether the entered email exists in the database. Then, it verifies if the user has confirmed their email and whether their account is active. The password provided by the user is compared with the stored hashed password using bcrypt for security. If authentication is successful, a JSON Web Token (JWT) is generated to maintain the user's session and the user's last active time is updated. The module then returns the user details along with the token for secure access to protected resources.

viii. User Authentication Module

Code Snipped:

```

import jwt from 'jsonwebtoken';

const authMiddleware = (req, res, next) => {
  const token =

```

```

req.cookies?.token ||
(req.headers.authorization && req.headers.authorization.split(' ')[1]);

if (!token) {
  return res.json({ success: false, message: 'token missing' });
}

try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = { _id: decoded.id, email: decoded.email };
  next();
} catch (err) {
  const message =
    err.name === 'TokenExpiredError' ? 'Token expired' : 'Invalid Token';
  res.status(403).json({ success: false, message });
}
};

export default authMiddleware;

```

This module ensures that only authenticated users can access protected routes within the system. It works by checking for a valid JSON Web Token (JWT) in either the request cookies or the Authorization header. If a token is present, it is verified using the server's secret key. Upon successful verification, the user's ID and email are attached to the request object, allowing the request to proceed to the next middleware or controller. If the token is missing, expired, or invalid, the module blocks access and returns an error message, thereby preventing unauthorized access and maintaining secure sessions.

ix. Cart Module

Code Snipped:

```
// Get cart
```

```

export const getCart = asyncHandler(async (req, res) => {
  const items = await CartItem.find({ user: req.user._id }).populate("item");

  console.log('getCart found items:', items.length);
  items.forEach((ci, idx) => {
    console.log(`Item ${idx}:`, {
      _id: ci._id,
      itemPopulated: !!ci.item,
      itemId: ci.item?._id,
      itemName: ci.item?.name,
      quantity: ci.quantity
    });
  });

  const formatted = items.map((ci) => ({
    _id: ci._id.toString(),
    item: ci.item,
    quantity: ci.quantity,
  }));

  res.json(formatted);
});

// Add to cart
export const addToCart = asyncHandler(async (req, res) => {
  const { itemId, quantity } = req.body;
  console.log('addToCart received:', { itemId, quantity, type: typeof quantity, body: req.body });

  if (!itemId || (quantity === undefined || quantity === null)) {
    res.status(400);
    throw new Error("itemId and quantity are required");
  }
}

```

```

const qty = Number(quantity);
if (!Number.isFinite(qty) || qty < 1) {
  res.status(400);
  throw new Error("quantity must be a valid number >= 1");
}

let cartItem = await CartItem.findOne({ user: req.user._id, item: itemId });

if (cartItem) {
  cartItem.quantity = Math.max(1, cartItem.quantity + qty);

  if (cartItem.quantity < 1) {
    await cartItem.remove();
    return res.json({
      _id: cartItem._id.toString(),
      item: cartItem.item,
      quantity: 0,
    });
  }

  await cartItem.save();
  await cartItem.populate("item");
  return res.status(200).json({
    _id: cartItem._id.toString(),
    item: cartItem.item,
    quantity: cartItem.quantity,
  });
}

cartItem = await CartItem.create({
  user: req.user._id,
  item: itemId,

```

```

    quantity: qty,
  });
  await cartItem.populate("item");

  console.log('Created cart item:', {
    _id: cartItem._id,
    itemPopulated: !!cartItem.item,
    itemName: cartItem.item?.name,
    itemPrice: cartItem.item?.price,
    quantity: cartItem.quantity
  });

  res.status(201).json({
    _id: cartItem._id.toString(),
    item: cartItem.item,
    quantity: cartItem.quantity,
  });
});

// Update cart item
export const updateCartItem = asyncHandler(async (req, res) => {
  const { quantity } = req.body;
  const cartItem = await CartItem.findOne({
    _id: req.params.id,
    user: req.user._id,
  });
  if (!cartItem) {
    res.status(404);
    throw new Error("cart item not found");
  }

  cartItem.quantity = Math.max(1, quantity);
  await cartItem.save();
});

```

```

    await cartItem.populate("item");

    res.json({
      id: cartItem._id.toString(),
      item: cartItem.item,
      quantity: cartItem.quantity,
    });
  });

// Delete cart item
export const deleteCartItem = asyncHandler(async (req, res) => {
  const cartItem = await CartItem.findOne({
    _id: req.params.id,
    user: req.user._id,
  });
  if (!cartItem) {
    res.status(404);
    throw new Error("cart item not found");
  }

  await cartItem.deleteOne();
  res.json({ _id: req.params.id });
});

// Clear cart
export const clearCart = asyncHandler(async (req, res) => {
  await CartItem.deleteMany({ user: req.user._id });
  res.json({ message: 'cart cleared' });
});

```

This module manages the shopping cart functionality for users. It allows authenticated users to add items to their cart, view all their cart items, update item quantities, and remove items when needed. Each cart record is tied to a specific user and item, ensuring personalized and secure cart operations.

Get Cart : This function collects all the items that the logged-in user has added to their cart. It also includes the item details (like name, price, etc.) so the cart can be shown clearly on the app/website.

Add to Cart: Adds a new item to the user's cart or increases quantity if the item already exists. It validates the input quantity to prevent invalid entries.

Update Cart Item: Allows the user to change the quantity of a specific item in their cart while ensuring quantity does not go below one.

Delete Cart Item: Removes a specific item from the user's cart.

Clear Cart: Deletes all cart items for the current user, typically used after checkout. This module ensures smooth cart operations, accurate item tracking, and a consistent shopping experience.

x: Item Module

Code Snipped:

```
import itemModel from "../models/itemModel.js";

// Create Item
export const createItem = async (req, res, next) => {
  try {
    const { name, description, category, price, rating, hearts } = req.body;
    const imageUrl = req.file ? `/uploads/${req.file.filename}` : "";

    const total = Number(price) * 1;

    const newItem = new itemModel({
      name,
      description,
```

```

    category,
    price,
    rating,
    hearts,
    imageUrl,
    total,
  });

  const saved = await newItem.save();
  res.status(201).json(saved);
} catch (err) {
  if (err.code === 11000) {
    res.status(400).json({ message: 'item name already exists' });
  } else {
    next(err);
  }
}
};

// Get all items
export const getItems = async (req, res, next) => {
  try {
    const items = await itemModel.find().sort({ createdAt: -1 });
    const host = `${req.protocol}://${req.get('host')}`;

    const withFullUrl = items.map(i => ({
      ...i.toObject(),
      imageUrl: i.imageUrl ? host + i.imageUrl : '',
    }));

    res.json(withFullUrl);
  } catch (err) {
    next(err);
  }
};

```

```

    }
  };

  // Delete item
  export const deleteItem = async (req, res, next) => {
    try {
      const removed = await itemModel.findByIdAndDelete(req.params.id);

      if (!removed) return res.status(404).json({ message: "item not found" });

      res.status(204).end();
    } catch (err) {
      next(err);
    }
  };

```

This module handles all operations related to managing menu items in the system. It allows administrators to create, retrieve, and delete items from the database.

Create Item:

This function receives item details along with an uploaded image and stores the new item in the database. It automatically calculates the total price and ensures that duplicate item names are not allowed.

Get All Items:

This function retrieves all items from the database and sends them to the frontend. It also formats the image URL so that it can be displayed properly in the application.

Delete Item:

This function removes an item from the database based on its ID. If the specified item does not exist, it returns a “not found” message.

4.3 Testing

Testing ensures that the system performs as intended, detects errors early, and minimizes maintenance costs. In the Web-Based Food Ordering System, testing was conducted to verify functionality, reliability, and performance across all modules, including user authentication, menu management, cart operations, payment processing, and recommendations. Manual and API testing (using Postman) were applied to confirm accurate data flow between the client, server, and database.

4.3.1 Unit Testing

Unit testing was performed after developing each module to verify its functionality in isolation. Tests were applied to backend functions such as authentication, menu CRUD, cart management, and order creation to ensure correct responses and error handling. Frontend components were also tested to confirm accurate data display and form validation. This process ensured that each unit worked correctly before integration, improving overall system stability and reducing future errors.

Table 2 :Test case for unsuccessful registration for user

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Test Data
001	Verify that all the required fields are present and filled.	Display an error message prompting the user to fill out all required fields.	Please fill out this field	pass	Username: Email: sishirgautam12@gmail.com Password: 12345678

Table 3 :Test case for successful registration for user

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/ Fail	Test Data
001	Verify that all the required fields are present and filled.	Registration successful! Please check your email for verification code.	Registration successful! Please check your email for verification code	Pass	Username: Shishir Email: sishirgautam12@gmail.com Password: 12345678 Otp:923451

Table 5: Test case for successful registration for user

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/ Fail	Test Data

001	Verify that the Otp is sent to the Email	Registration successful! Please check your email for verification code.	Registration successful! Please check your email for verification code	Pass	Username: Shishir Email: sishirgautam12@gmail.com Password: 12345678 Otp:923451
-----	--	--	---	------	---

Table 4 :Test case for unsuccessful login for user

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/ Fail	Test Data
001	Verify that the system displays an error for incorrect email.	" User doesn't exist."	"User doesn't exist "	Pass	Username: Shishir Password: 12345678
002	Verify that the system displays an error for incorrect password.	" Invalid password."	"Invalid password"	Pass	Username: Password: 12345678

Table 5 :Test Case for successful login for user

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Test Data
001	Verify that the system allows login with correct username and password.	Login successful	Login successful	Pass	Username: Shishir Password: 12345678

Table 6 :Test Case for successful registration for admin

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Test Data
001	Verify that all the required fields are present and filled.	Pending Owner Approval Registration request sent! Please wait for owner approval. You will receive an email once approved.	Pending Owner Approval Registration request sent! Please wait for owner approval. You will receive an email once approved.	Pass	Username: Shishir Email Address: sishiruata12@gmail Password: 12345678 Confirm Password: 12345678 OTP=736890

004	Approve the user registration by Admin	Admin Approved Successfully!	Admin Approved Successfully!	Pass	
-----	--	------------------------------	------------------------------	------	--

Table 7 :Test Case for unsuccessful registration for admin

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Test Data
003	Verify that the system displays an error for invalid email.	Please enter a valid email	Please enter a valid email	Pass	Username: Shishir Email Address: sishiruata12@gmail Password: 12345678 Confirm Password: 12345678
004	Verify that the system displays an error for invalid password.	Passwords do not match	Passwords do not match	Pass	Username: Shishir Email Address: sishirgautam12@gmail Password: 12345678 Confirm Password: 12345678

Table 8 :Test Case for unsuccessful login for admin

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Test Data
003	Verify that all the required fields are present and filled.	Display an error message prompting the user to fill out all required fields.	Please fill out this field.	Pass	Email Address: Password: 12345678
004	Verify that the system displays an error for invalid email id.	Admin account doesn't exist	Admin account doesn't exist	Pass	Email Address: sishirgautm12@gmail.com Password: 12345678
005	Verify that the system displays an error for invalid password.	Invalid password	Invalid password	Pass	Email Address: sishirgautm12@gmail.com Password: 1234567

Table 9 :Test Case for successful login for admin

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Test Data
---------------------	------------------------------	------------------------	----------------------	------------------	------------------

003	Verify that all the required fields are present and filled.	Login successful	Login successful	Pass	Email Address: sishirgautm12@gmail.com Password: 12345678
-----	---	------------------	------------------	------	---

4.3.2 System Testing

System testing was conducted to evaluate the overall functionality, integration, and performance of the TrioOrder Food Ordering System as a complete and unified application. The main objective of this phase was to ensure that all modules including user registration and login, menu management, order placement, online payment processing, recommendation generation, and admin operations work together correctly and meet the system's requirements. The testing involved several methods such as functional testing, integration testing, performance testing, security testing, and usability testing. Functional testing confirmed that each feature operated according to specifications, while integration testing verified smooth communication between the frontend, backend, database, and external services such as the Stripe payment gateway and Nodemailer (Gmail SMTP). Security testing ensured that sensitive data like user credentials and payment information were protected through JWT authentication and encryption. Performance and usability testing demonstrated that the system was responsive, stable, and user-friendly across different devices and browsers. Overall, the results of system testing indicated that the TrioOrder application met all functional and non-functional requirements, operated reliably under expected conditions, and was ready for deployment with only minor optimizations identified and resolved during the process.

Table 10 :Test case for System Testing of admin panel

SN	Test Case	Steps	Test Data	Expected Result	Pass/ Fail
----	-----------	-------	-----------	-----------------	---------------

001	Registration Page	Go to registration page	Username: Shishir Email Address: sishiruata12@gmail Password: 12345678 Confirm Password: 12345678 Otp:745356	Registration successful! You can now log in	Pass
002	Verify Login	1. Go to login page 2. Enter correct Email and Password 3. Click on login button	Email Address: sishirgautm12@gmail.com Password: 12345678	Login Successful	Pass
003	Verify if the admin can add new menu item	1. Go to add items 2. Upload item image, enter product name, description, price and select category 3. Click on button add to menu	Product name: Tea Description: Hot brewed drink with a refreshing and comforting taste. Category: drinks Price: 40	'Item Added Successfully!' 'New menu item has been created', 'The item is now visible to customers.'	Pass

004	Add Special offer	1. Go to special menu page 2. Click on Add Special Offer 3. Upload image, enter Title, description, select item, input original price & discounted price, valid date, priority and tags 4. Click on create order	Title: Tea Description: Hot brewed drink with a refreshing and comforting taste. Select Item: Tea Original Price: 40 Discounted Price: 20 Discount Percentage: 50 Valid Until:11/12/2025 Priority (0-10): 1	Special Offer Saved! New special offer created successfully The offer is now active and visible to customers.	Pass
005	Update order status	1. Click on Orders from navigation bar 2. Toggle in action section of order list 3. Select the appropriate status (Pending, preparing,	Pending, preparing, processing, on the way or cancelled	Order Status Updated!	Pass

		processing, on the way or cancelled)			
006	Activate or Deactivate user	1. Click on Users from navigation bar 2. Toggle to the action column of user list 3. Click on deactivate or activate respectively to deactivate or active users	Deactivate or Activate	User deactivated successfully! Or User activated successfully!	Pass
007	Update contact queries status	1. Click on Contact queries from navigation bar 2. Click on view detail (eye icon) in action row of queries list	Mark In Progress, Mark Resolved or Close Query	Query status updated successfully	Pass

		3. Select Status & Priority according to condition i.e. Mark In Progress, Mark Resolved or Close Query			
008	Logout	1. Click on button Logout in navbar		Logout Successful	pass

Table 11: Test case for System Testing of User panel

SN	Test Case	Steps	Test Data	Expected Result	Pass/Fail
001	User Registration	1. Go to registration page 2. Click on create new account 3. Enter Username, Email and password	Username: Shishir Email: sishirgautam12@gmail.com Password: 12345678	Registration successful! You can now log in	Pass

		4. Click on sign up button			
002	User verification	Enter OTP	Received OPT in email	Registration successful	pass
003	User Login	1. Go to login page 2. Enter email and password 3. Click on login button	Email address: Sishirgautam12@gmail.com Password: 12345678	Login successful	Pass
004	Add to cart	1. Go to menu page from navbar 2. Select food items by clicking add to cart button inside the food card	Test data: Chicken Roll, English Breakfast, kheer ,Fruit salad		pass

005	View cart and view Recommendation	1. Click on cart icon in navbar 2. Scroll down to view recommended items	Recommend: Buffalo Wings , Tomato Soup, Mix veg Salad , Grilled Chicken		pass
006	Proceed to checkout	1. In cart page scroll down 2. Click on Proceed to checkout button 3. Enter personal information 4. i.e. first name, last name, phone, email, address, city, zip code, 5. select payment method 6. click on complete order	First Name: Shishir Last Name: Gautam Phone: 9317862456 Email: shishir@gmail.com Address: ABC Pvt. Ltd. Attn: Finance Department P.O. Box 789 City: Kathmandu Zip Code: 44600	Redirected to order history	

007	Confirm order delivered	After admin update status of order to “on the way” 1. Go to My Orders from navbar 2. Click on Confirm Delivery from action row of list of orders 3. Click Confirm delivery in popup query		Delivered	Pass
008	Submit Query	1. Go to contact page from navbar 2. Enter full name, phone number, Email address, Address, Dish name, your Query 3. Enter submit button	Full Name: Shishir Gautam Phone Number: +9779317862456 Email Address: shishir@gmail.com Address: ABC Pvt. Ltd. Attn: Finance Department P.O. Box 789 Dish Name: Tea Your Query: good Offer	Your query has been submitted successfully. We will get back to you soon!	Pass

4.4 Result Analysis

The food recommendation system in our Web-Based food Ordering System was trained using 60 delivered orders, applying the Apriori algorithm with a minimum support threshold of 0.05 and a minimum confidence threshold of 0.30. These parameters ensured that only meaningful and frequently occurring item combinations were selected for rule generation. With these thresholds, the algorithm generated 33,692 association rules, from which the system extracts the top matching rules for a given user's cart

For evaluation, the test cart contained English Breakfast, Chicken Roll, Fruit Salad, and Kheer. The system generated five recommendations, each supported by real patterns discovered in historical customer orders. Manual verification of the association rules confirmed the system's calculations:

- Buffalo Wings and Tomato Soup achieved a confidence of 1.00, meaning they appeared in every order that contained the respective antecedent combinations.
- Mix Veg Salad and Grilled Chicken both yielded 0.75 confidence, occurring in three out of four orders where the antecedent appeared.
- Chicken Chilly achieved 0.67 confidence, included in two out of three orders that contained the antecedent set of English Breakfast, Chicken Roll, and Fruit Salad.

Relevant Orders Used in the Confidence Calculations

To validate the association rules produced by the Apriori algorithm, it is necessary to examine only those historical orders that meaningfully contribute to each rule. Although the total dataset contained 60 delivered orders, the Apriori algorithm does not use all orders equally. Instead, it applies minimum support filtering, which determines which item combinations (itemsets) occur frequently enough to be considered meaningful.

In this project, the minimum support was set to: $\text{minSupport} = 0.05$. Given 60 total orders, an itemset must appear at least: $60 \times 0.05 = 3$ orders to be retained as a frequent itemset. Any item combination appearing in fewer than three transactions is pruned and excluded from rule generation. Because of this filtering process, only a small subset of orders specifically those containing repeated combinations such as English Breakfast, Chicken Roll, Fruit Salad, Kheer, Buffalo Wings, Tomato Soup, and Mix Veg Salad passed the support threshold. These orders collectively contain the multi-item patterns

that occur at least three times across the dataset. As a result, they form the relevant transactional evidence used to compute confidence for the final recommendations.

The following five orders met the minimum support threshold for the itemsets associated with the user's cart (English Breakfast, Chicken Roll, Fruit Salad, and Kheer), and therefore constitute the basis for all confidence calculations:

Order 691279739156c0d5834f39e4: Chicken Momo, English Breakfast, Grilled Chicken, Fruit Salad, Buffalo Wings, Tomato Soup, Kheer, Americano Cappuccino.
 Order 691279b49156c0d5834f3b51: English Breakfast, Chicken Roll, Fruit Salad, Buffalo Wings, Tomato Soup, Kheer, Americano Cappuccino.
 Order 691279ef9156c0d5834f3cd3: English Breakfast, peanut butter toast, Grilled Chicken, Chicken Roll, Chicken fried rice, Mix Veg Salad, Fruit Salad, Chicken Chilly, Buffalo Wings, Mix Veg Soup, Tomato Soup, Chocolate Cake, Kheer, Tea.
 Order 69127a439156c0d5834f3e74: English Breakfast, Omelette, pancake, Chicken Roll, Grilled Chicken, Pizza (Chicken), Mix Veg Salad, Fruit Salad, Chicken Chilly, Buffalo Wings, Chicken Momo, Mix Veg Soup, Chocolate Cake, Chocolate Shake.
 Order 69127aa29156c0d5834f4025: English Breakfast, overnight oats, Omelette, pancake, Grilled Chicken, Chicken Roll, Pizza (Chicken), Mix Veg Salad, Egg Salad, Chicken Momo, Yomari, Buffalo Wings, Tomato Soup, Chicken Soup, Kheer, Chocolate Brownie.

Association Rule Analysis: For each rule, the antecedent (A), consequent (B), relevant orders, support counts, and confidence values are formally examined. All calculations are based on the subset of orders containing the items selected in the user's cart: English Breakfast, Chicken Roll, Fruit Salad and Kheer.

Rule 1 – Buffalo Wings

(English Breakfast + Chicken Roll $\rightarrow$ Buffalo Wings)

Antecedent (A): {English Breakfast, Chicken Roll}

Consequent (B): {Buffalo Wings}

Support and Confidence: The support count for the antecedent A is 4/60 transactions. The support count for A $\cup$ B is also 4/60, indicating that every occurrence of English Breakfast, Chicken Roll together is accompanied by Buffalo Wings.

Support(English Breakfast, Chicken Roll)

$$= \frac{\text{Number of transactions containing English Breakfast, Chicken Roll}}{\text{Total number of transactions}}$$

$$\text{Support}(A) = \frac{4}{60} = 0.067 \text{ (6.7\%)}$$

So support 0.067 (6.7%) This meets your minimum support threshold 0.05 (5%) .So the itemset is accepted.

$$\text{Confidence}(\text{English Breakfast, Chicken Roll} \rightarrow \text{Buffalo Wings}) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

$$\text{Confidence}(A \rightarrow B) = \frac{4/60}{4/60} = 1.00$$

This rule has a perfect confidence score of 1.00, matches the system's computed value.

Rule 2 – Tomato Soup

(English Breakfast + Kheer $\rightarrow$ Tomato Soup)

Antecedent (A): {English Breakfast, Kheer}

Consequent (B): {Tomato Soup}

Support and Confidence: The support count for A is 4/60 transactions. The support count for A $\cup$ B is also 4/60, showing that all orders containing English Breakfast and Kheer also contain Tomato Soup.

$$\text{Support}(A) = \frac{4}{60} = 0.067 \text{ (6.7\%)}$$

This exceeds the minimum support threshold of 5%.

$$\text{Confidence}(A \rightarrow B) = \frac{4/60}{4/60} = 1.00$$

Thus, the rule has a confidence score of 1.00, consistent with the system output

Rule 3 – Mix Veg Salad

(English Breakfast + Chicken Roll $\rightarrow$ Mix Veg Salad)

Antecedent (A): {English Breakfast, Chicken Roll}

Consequent (B): {Mix Veg Salad}

Support and Confidence: The support count for A is 4/60 transactions. The support count for A $\cup$ B (orders that contain English Breakfast, Chicken Roll, *and* Mix Veg Salad) is 3/60 transactions.

$$\text{Support}(A) = \frac{4}{60} = 0.067$$

$$\text{Support}(A \cup B) = \frac{3}{60} = 0.05$$

This meets the 5% minimum support requirement.

$$\text{Confidence}(A \rightarrow B) = \frac{3/60}{4/60} = \frac{3}{4} = 0.75$$

Thus, this rule has a confidence value of 0.75, matching the system's output.

Rule 4 – Grilled Chicken

(English Breakfast + Chicken Roll $\rightarrow$ Grilled Chicken)

Antecedent (A): {English Breakfast, Chicken Roll}

Consequent (B): {Grilled Chicken}

Support and Confidence: The support count for A is 4/60 transactions. The support count for A $\cup$ B is 3/60 transactions, meaning that three of the four orders containing English Breakfast and Chicken Roll also contain Grilled Chicken.

$$\text{Support}(A) = \frac{4}{60} = 0.067$$

$$\text{Support}(A \cup B) = \frac{3}{60} = 0.05$$

This satisfies the minimum support threshold.

$$\text{Confidence}(A \rightarrow B) = \frac{3/60}{4/60} = \frac{3}{4} = 0.75$$

This rule therefore has 0.75 confidence, identical to the system-generated score.

Rule 5 – Chicken Chilly

(English Breakfast + Chicken Roll + Fruit Salad $\rightarrow$ Chicken Chilly)

Antecedent (A): {English Breakfast, Chicken Roll, Fruit Salad}
Consequent (B): {Chicken Chilly}

Support and Confidence: The support count for A is 3/60 transactions (only three orders contain all three items together). The support count for A ∪ B is 2/60 transactions, meaning two of those three orders also include Chicken Chilly.

$$\text{Support}(A) = \frac{3}{60} = 0.05$$

$$\text{Support}(A \cup B) = \frac{2}{60} = 0.033$$

Both values meet the Apriori support criteria.

$$\text{Confidence}(A \rightarrow B) = \frac{2/60}{3/60} = \frac{2}{3} \approx 0.67$$

The resulting confidence score (0.67) matches the system's computed value.

System Ranking for the Recommended items based on Confidence Scores

Using the confidence values calculated for each rule, the system ranks all recommended items from highest to lowest confidence. In this case, both Buffalo Wings and Tomato Soup achieved perfect confidence scores of **1.00**, meaning they appeared every time their corresponding antecedent item combinations occurred, so they are placed at the top of the list. Items with slightly lower confidence values, such as Mix Veg Salad and Grilled Chicken (each **0.75**), appear next because their associations, while strong, are not as consistently observed. Finally, Chicken Chilly, with a confidence score of approximately **0.67**, is ranked last among the five recommendations.

1. Buffalo Wings (1.00)
2. Tomato Soup (1.00)
3. Mix Veg Salad (0.75)
4. Grilled Chicken (0.75)
5. Chicken Chilly (0.67)

The analysis demonstrates that the system is able to accurately identify strong and meaningful item associations from historical food orders. By applying minimum thresholds for support and confidence, the system ensures that only reliable and behaviour-backed

rules are used for generating recommendations. The calculated confidence scores clearly explain why certain items such as Buffalo Wings and Tomato Soup consistently rank highest, while others with moderate confidence values follow. Overall, the results confirm that the system ranks recommendations in a logical and data-driven manner, prioritising items that have repeatedly appeared together in past transactions. This supports the effectiveness of the algorithm in delivering relevant and trustworthy suggestions to enhance the user's ordering experience.

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

The development and implementation of the PetPuja Food Ordering System successfully achieved its goal of creating a modern, user-friendly, and intelligent platform for restaurants and customers. Throughout the design, coding, and testing stages, the system demonstrated strong performance, secure operations, and seamless integration between all modules. Users were able to browse menus, place orders, make online payments, and receive real-time updates with ease, while the admin could efficiently manage items, track orders, view analytics, and update recommendations.

One of the most impactful features of the system was the smart recommendation engine, powered by the Apriori algorithm, which analyzed previous historic orders to provide food suggestions. This not only enhanced the user experience but also supported restaurants in increasing sales through relevant recommendations. The integration of Stripe API for secure payments and Nodemailer for instant email notifications further strengthened the system's reliability and professionalism. The System proved to be a practical and scalable solution that simplifies restaurant management and online ordering.

5.2 Future Recommendations

While the PetPuja Web-Based Food Ordering System has achieved its primary goals of providing a secure, efficient, and user-friendly platform for both customers and administrators, there is still room for future enhancement and innovation. The following recommendations outline potential improvements that could further strengthen and expand the system:

Enhanced Recommendation Model: Incorporate more advanced machine learning algorithms, such as collaborative filtering or deep learning based models, to improve recommendation accuracy.

Mobile Application Integration: Extend the platform to mobile devices (Android/iOS) for increased accessibility and convenience.

Real-Time Order Tracking: Implement real-time delivery tracking using location-based services (e.g., Google Maps API).

Improved Security Measures: Strengthen data encryption, implement two-factor authentication, and enhance API security to ensure safer transactions.

Cloud Deployment and Scaling: Deploy the system using cloud platforms such as AWS or Azure for better performance and load balancing under high traffic.

Multilingual Support: Add multi-language interfaces to make the platform more inclusive and user-friendly.

Loyalty and Reward System: Introducing a points-based loyalty program can encourage repeat orders by rewarding customers for frequent purchases or referrals, thereby improving customer retention

By implementing these recommendations, the system can evolve into a more intelligent, secure, and scalable platform that meets the growing demands of modern food service industries.

REFERENCES

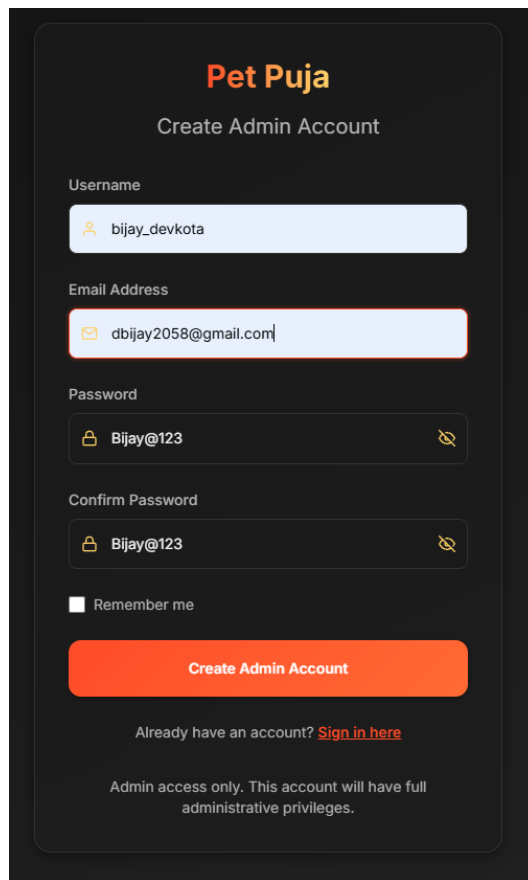
- [1] T. Sunitha, S. Amancha, V. V. S. DurgaDeepika, N. Akhil Sai Vamsi, P. Alekhya, P. BhagyaSri, T. Likhitha Sri, and Y. V. S. N. Hanush, "Paper presented at the International Conference on Recent Advances in Information Technology," International Conference on Recent Advances in Information Technology (RAIT), 2025. [Online]. Available: <https://doi.org/10.1109/RAIT65068.2025.11088956>, [Accessed: May 2025].
- [2] T. Sunitha, S. Amancha, V. V. S. DurgaDeepika, N. Akhil Sai Vamsi, P. Alekhya, P. BhagyaSri, T. Likhitha Sri, and Y. V. S. N. Hanush, "Paper presented at the International Conference on Recent Advances in Information Technology," International Conference on Recent Advances in Information Technology (RAIT), 2025. [Online]. Available: <https://doi.org/10.1109/RAIT65068.2025.11088956>, [Accessed: May 2025].
- [3] D. Subedi, M. Paudel, S. Poudel, and N. Koirala, "Food Safety in Developing Countries: Common Foodborne and Waterborne Illnesses, Regulations, Organizational Structure, and Challenges of Food Safety in the Context of Nepal," *Food Frontiers*, vol. 6, no. 1, pp. 86–123, Jan. 2025. [Online]. Available: <https://doi.org/10.1002/fft2.517>. [Accessed: Aug. 31, 2025].
- [4] T. Kashima, S. Matsumoto, and H. Ishii, "Feasibility of Integrated Menu Recommendation and Self-Order System for Small-Scale Restaurants," *2010 AIP Conference Proceedings*, 2010. [Online]. Available: <https://doi.org/10.1063/1.3510540>, [Accessed: May 2025].
- [5] A. Fadhil, K. Al Jufri, S. A. Paskalis, and N. Rukhviyanti, "Design of a Web-Based Regional Food Ordering Information System at Seribu Rasa Restaurant," *Jurnal INOVTEK Polbeng – Seri Informatika*, vol. 10, no. 1, pp. 401–410, 2025. [Online]. Available: <https://doi.org/10.35314/mb5xe359>, [Accessed: Aug. 2025]
- [6] N. M. Yawale, N. V. Pardakhe, M. A. Deshmukh, and N. A. Deshmukh, "A Review Paper on Online Restaurant Management System," 2017. [Online]. Available: <https://www.semanticscholar.org/paper/A-Review-Paper-on-Online-Restaurant-Management-M.Yawale-Pardakhe/1042cba5d9b98b6746f6a3a79c3e147aa9cbd80f> [Accessed: Aug. 2025].

[7] S. Dasgupta and S. Karmakar, "Food Recommendation using Classifier and Modified Apriori Algorithm," *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, no. 12, pp. 3967–3970, Oct. 2019. [Online]. Available: <https://doi.org/10.35940/ijitee.I3471.1081219>. [Accessed: Aug. 31, 2025].

[8] E. Sheryl Kimes, “The Current State of Online Food Ordering in the U.S. Restaurant Industry,” 2011. [Online]. Available: <https://www.semanticscholar.org/paper/The-Current-State-of-Online-Food-Ordering-in-the-Kimes-Sheryl/af9d2cd161d7c8cb82bbbf19d746e443f8923aa> , [Accessed: Aug. 2025]

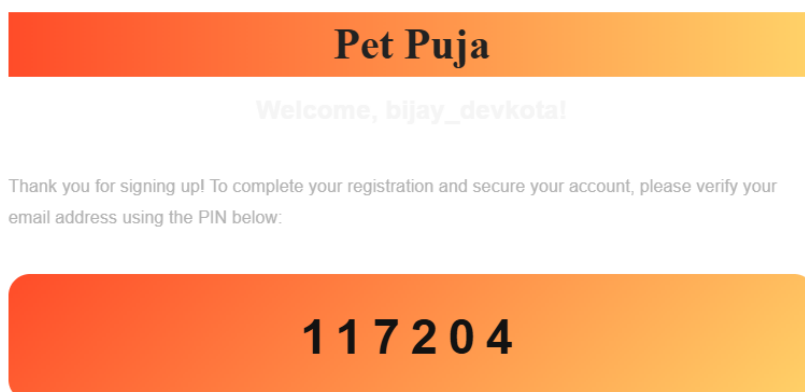
[9] R. Patil, A. Kale, and T. Patil, "Wireless Customizable Food Ordering System for a Restaurant Using Apriori and K-means Algorithm," *Imperial Journal of Interdisciplinary Research*, vol. 1, Apr. 2017, [Online]. Available: <https://www.semanticscholar.org/paper/Wireless-Customizable-Food-Ordering-System-for-a-Patil-Kale/de4a6ac6d61591a3eb5e1ccb3b14faadbefeddc1>, [Accessed: Aug. 2025].

APPENDIX- SCREENSHOT



The screenshot shows a dark-themed registration form for 'Pet Puja'. The title 'Pet Puja' is in orange, and 'Create Admin Account' is in white. The form includes fields for Username (bijay_devkota), Email Address (dbijay2058@gmail.com), Password (Bijay@123), and Confirm Password (Bijay@123). There is a 'Remember me' checkbox and a 'Create Admin Account' button. A link 'Sign in here' is provided for existing users. A disclaimer at the bottom states: 'Admin access only. This account will have full administrative privileges.'

Figure 7: Admin Registration



The screenshot shows a light-themed verification screen for 'Pet Puja'. The title 'Pet Puja' is in bold black. Below it, 'Welcome, bijay_devkota!' is in grey. A message says: 'Thank you for signing up! To complete your registration and secure your account, please verify your email address using the PIN below:'. A large orange box displays the PIN '117204'.

Figure 8 : OTP to verify Email

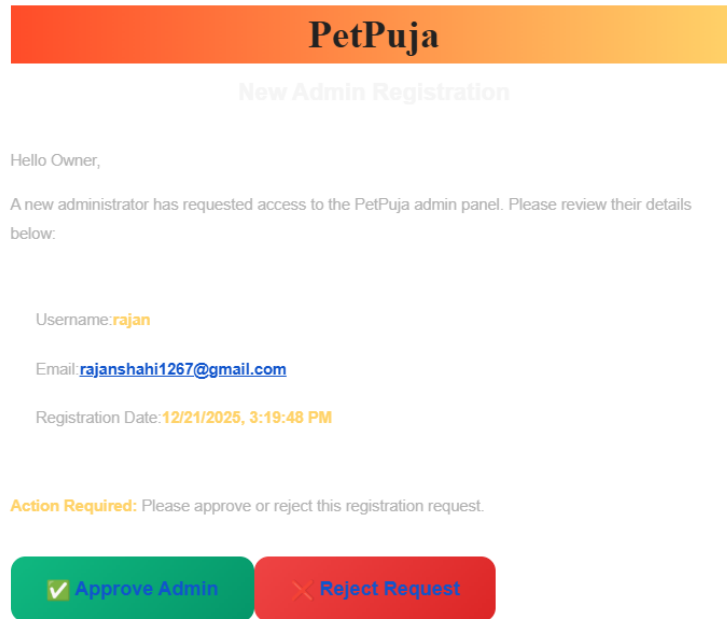


Figure 9 : Admin Approval Request

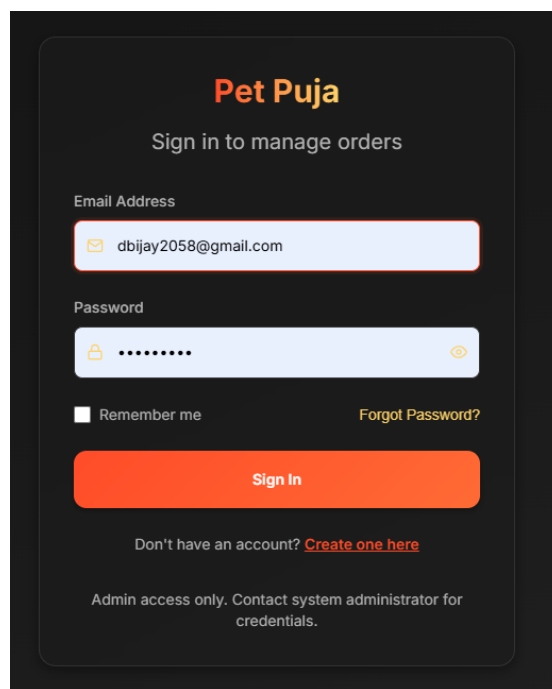


Figure 10: Admin Login

Add New Menu Item



Product Name *

Description *

Category *

Drinks

Price (₹) *

Rating

★
★
★
★
★
5/5

Popularity (Likes)

♥

Add To Menu

Figure 11: Add New Menu Item

Image	Name	Category	Price	Rating	Hearts	Delete
	Chicken ceaser Chicken ceaser salad	Snacks	₹554.98	★★★★★	♥ 0	
	Banana Toast Bread Toast with banana	Breakfast	₹120	★★★★★	♥ 0	
	Caramel Mochhito Caramel Mochhito	Desserts	₹332.98	★★★★★	♥ 0	
	Lava Cake Chocolate lava cake	Desserts	₹480	★★★★☆	♥ 0	
	Cheese cake Cheese cake with strawberry	Desserts	₹349.98	★★★★★	♥ 0	
	Chocolate Cake Cake with chocolate	Desserts	₹1199.99	★★★★★	♥ 0	
	Chicken Salad Grilled chicken with salad	Snacks	₹899.99	★★★★☆	♥ 0	

Figure 12: Manage New Menu Item

Special Offers

Title
Christmas Offer

Description
Enjoy banana toast

Select Item
Banana Toast - ₹120

Figure 13: Add special items

PetPuja Add Items List Items Special Offers **Orders** Statistics Users Contact Queries dbijay2058@gm... Logout

Order Management System

Manage customer orders efficiently

3 Total Orders 3 Active Orders 0 Delivered 0 Cancelled

Search by order ID, customer name, email, or phone... All Status Refresh

Order ID	Customer	Contact	Address	Total	Status	Actions
#F0599854 21 December 2022 at 04:42 pm	Bijay Devkota dbijay2058@gmail.com	9866338344	Jwagal, Lalitpur - 3234	₹325.00	Pending	Pending
#F059844a 21 December 2022 at 04:18 pm	Bijay Devkota dbijay2058@gmail.com	9866338344	Jwagal, Lalitpur - 418432	₹265.00	Pending	Pending
#F05982ee 21 December 2022 at 04:10 pm	Bijay Devkota dbijay2058@gmail.com	9866338344	Jwagal, Lalitpur - 2345	₹300.00	Pending	Pending

Figure 14: Order Management

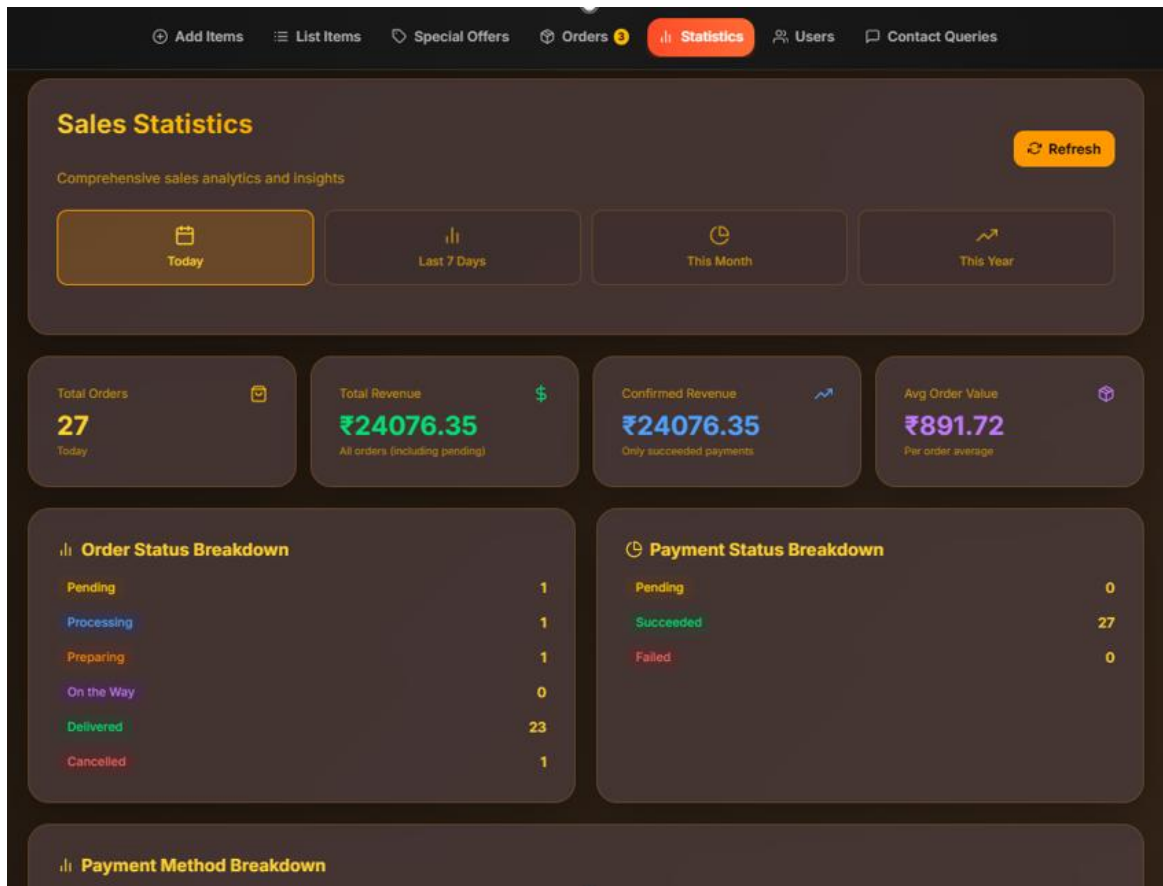


Figure 15: Sales Statistics

The "Query Details" modal contains the following sections:

Customer Information

Name: Bijay Devkota
Email: dbijay2058@gmail.com
Phone: +9779866338344
Address: Jwagal,Lalitpur
Dish: Rice

Query

wdefrg

Status & Priority

pending medium

Buttons: Mark In Progress, Mark Resolved, Close Query

Figure 16: Query Details

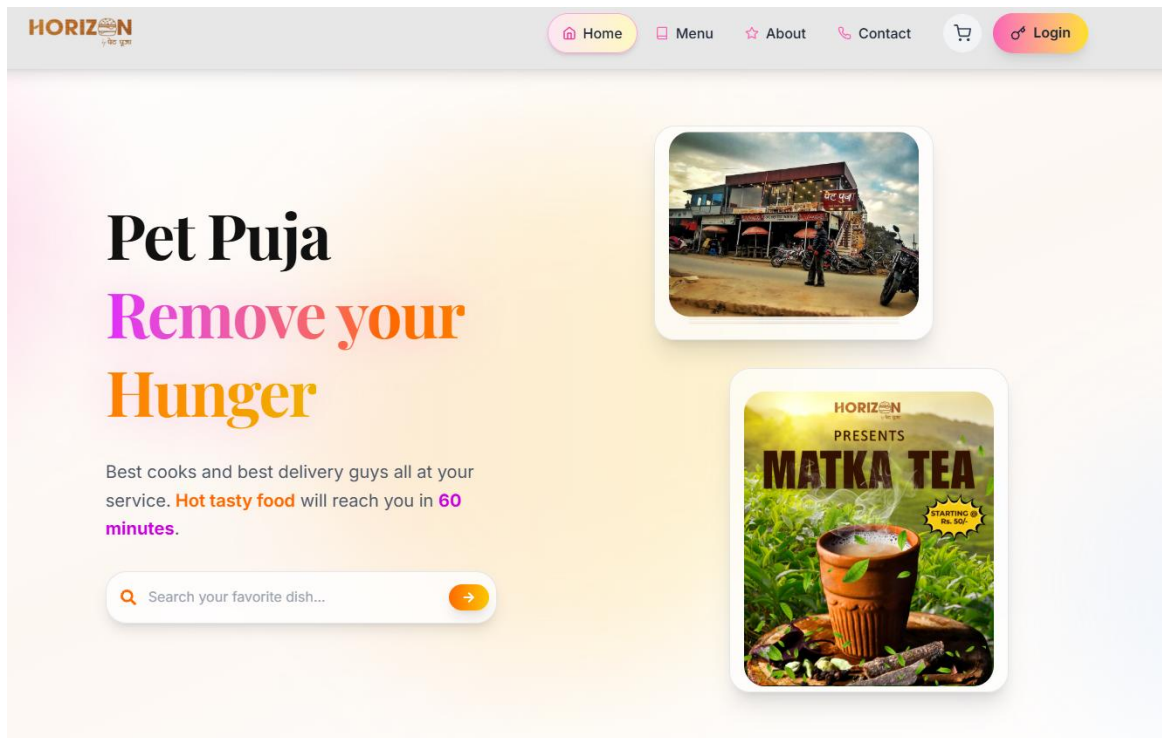


Figure 17: Home Page

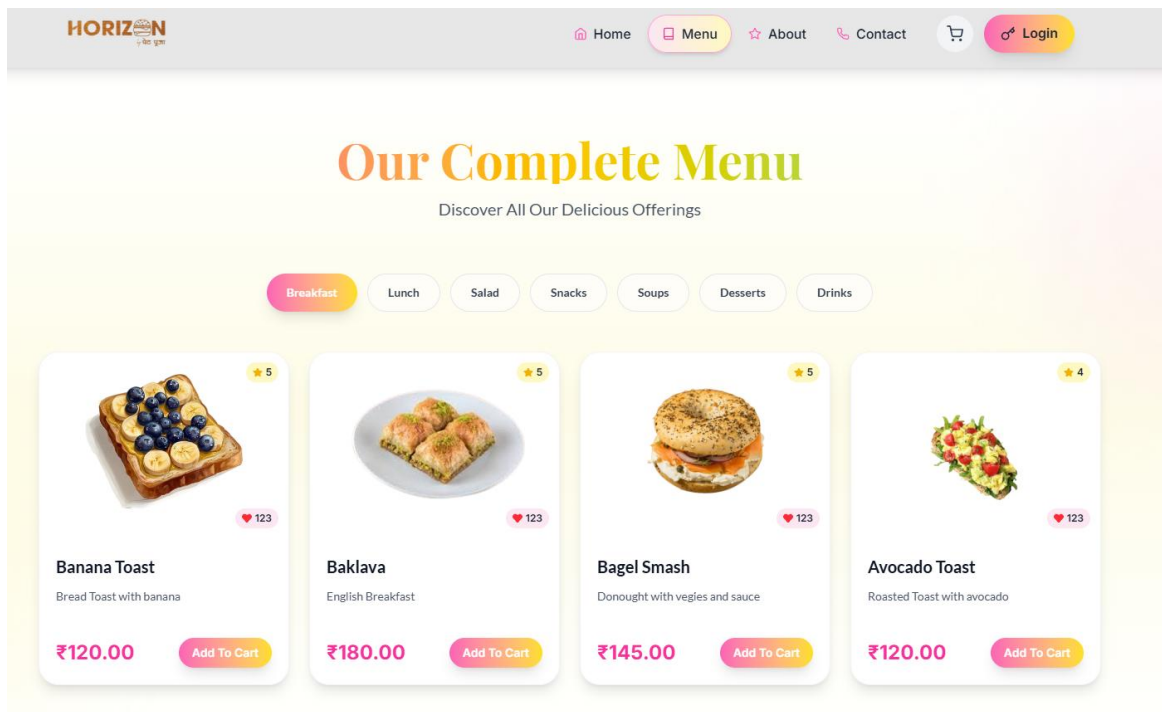


Figure 18: Menu

[← Back to Cart](#)

Checkout

Personal Information

First Name

Bijay

Last Name

Devkota

Phone

9866338344

Email

dbijay2058@gmail.com

Address

Jwagal,Lalitpur

City

Lalitpur

Zip Code

Figure 19: Checkout